

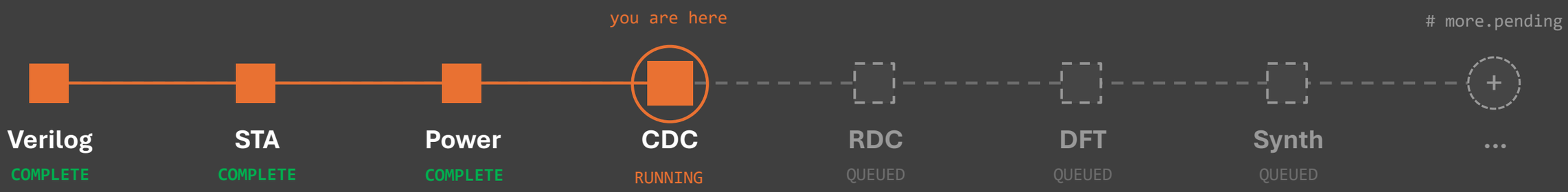
CDC

ChipRun // Digital-design

Episode 04 — CDC

Phase 02

It's been a long run,



But We haven't $arrived yet.

GitHub
code examples

Drive
full files and slides

LinkedIn
new episode alerts

Topics

Topics

What We Will Cover

1. Introduction
2. Metastability
3. MTBF
4. Synchronizers
5. Required Pulse Width
6. CDC Rules
7. Passing Events
8. Multi-bit: Gray Coding
9. Multi-bit: MUX Recirculation & MCP, Handshake
10. Asynchronous FIFO
11. Timing Constraints
12. Verification & Summary

Introduction

Introduction

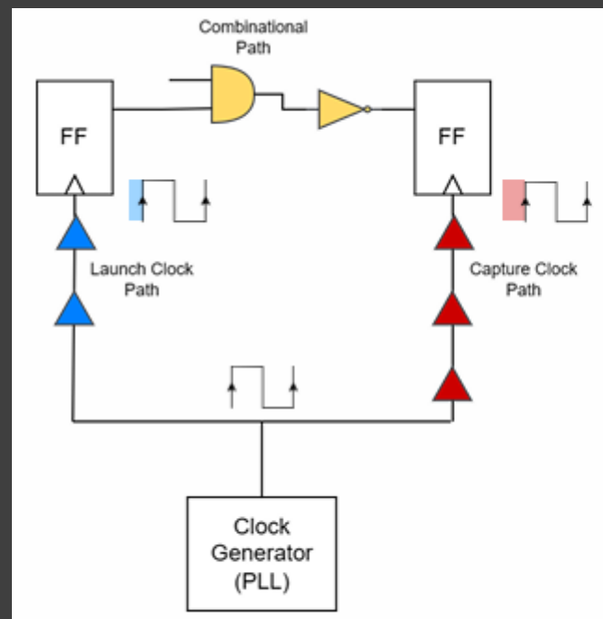
What is CDC?

- **Clock Domain Crossing: a signal traveling between two clock domains.**
- A clock domain is a group of registers clocked by the same clock.
- Real chips have many domains: different PLLs, core vs peripheral clocks, gated clocks.
- Signals crossing between asynchronous domains can be sampled at any phase — metastability and data corruption follow.
- **CDC issues are a primary cause of chip failure — this deck is about handling them systematically.**

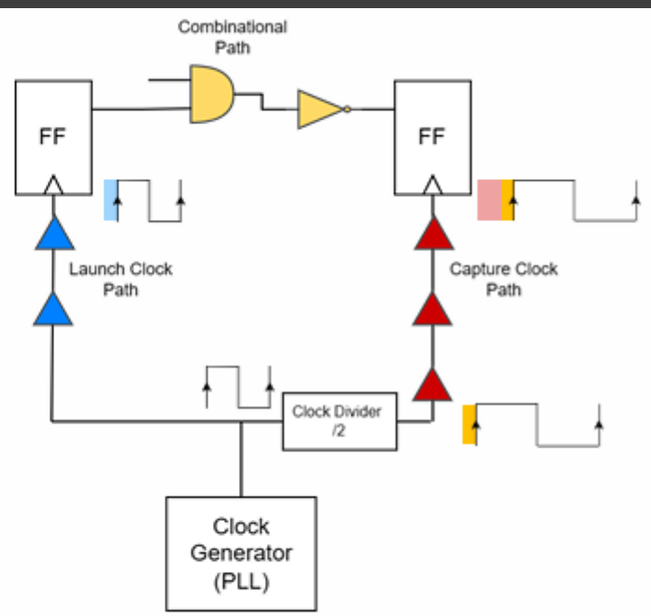
Introduction

Synchronous vs Asynchronous Clocks

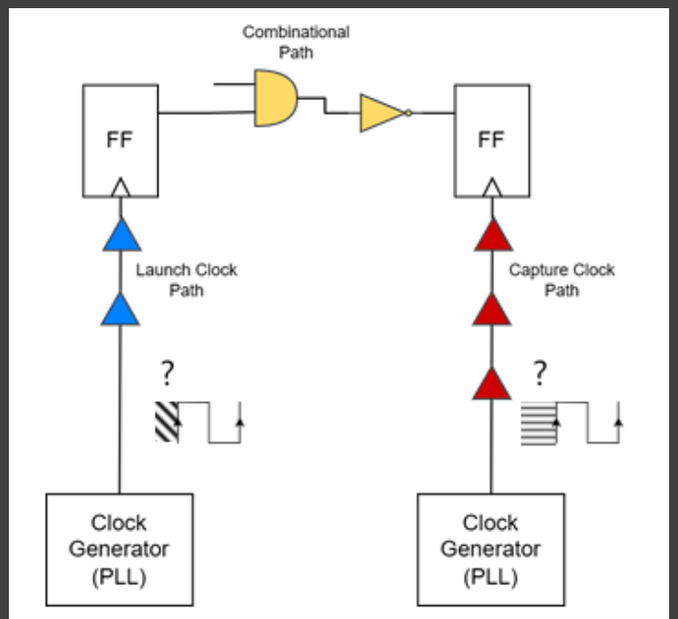
- **Synchronous clocks:** the phase difference is KNOWN — a clock and its divided/rational version of it.
- **Asynchronous clocks:** different sources (two PLLs, an oscillator) — phase difference UNKNOWN, even at the same frequency.
- STA can handle synchronous domains, including divided clocks (the divider delay is known).
- Between two independent PLLs nothing is known — that is where CDC starts.



Known Phase Difference



Known Phase Difference



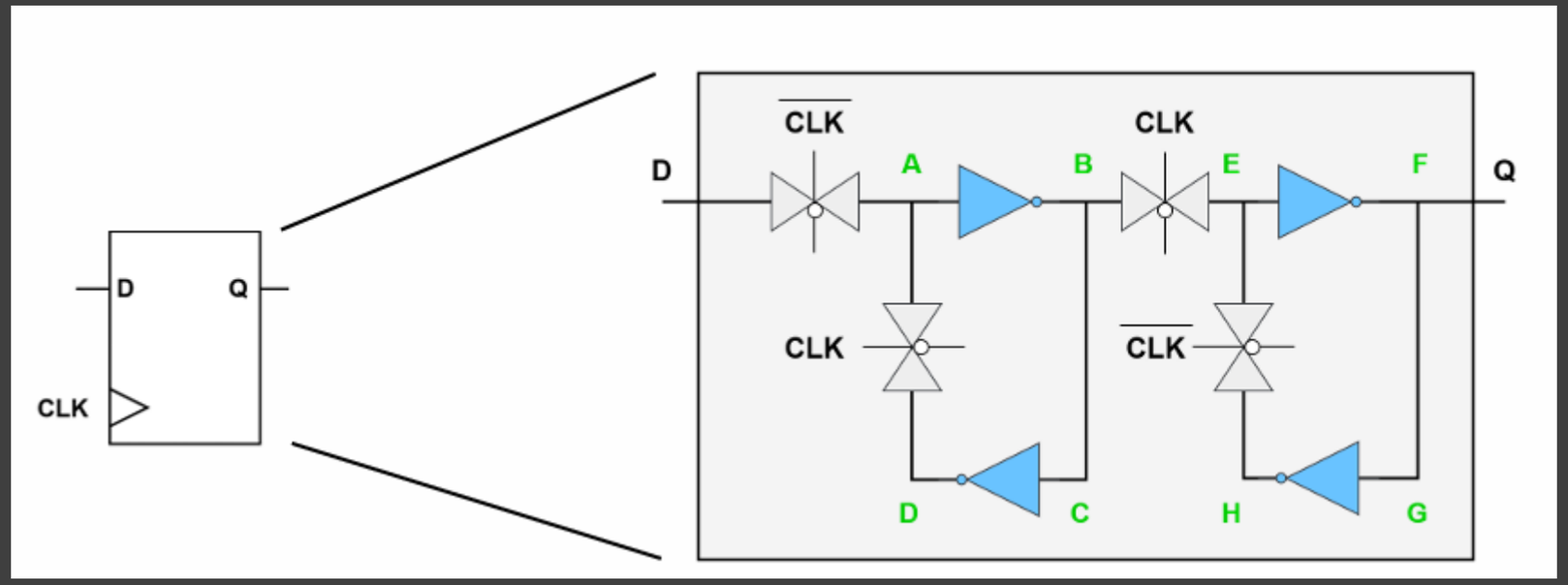
Unknown Phase Difference

Metastability

Metastability

Inside a Flip-Flop

- A D-FF can be built from inverters + transmission gates.
- The transmission gates are switches; the inverter loops are the storage elements.
- **CLK = 0:** input path open (D-A-B-C-D), master loop samples and waits.
- **CLK = 1:** the sampled value flows through B-E-F to the output pin Q.

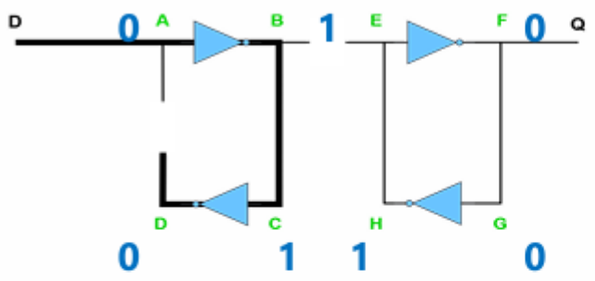


Metastability

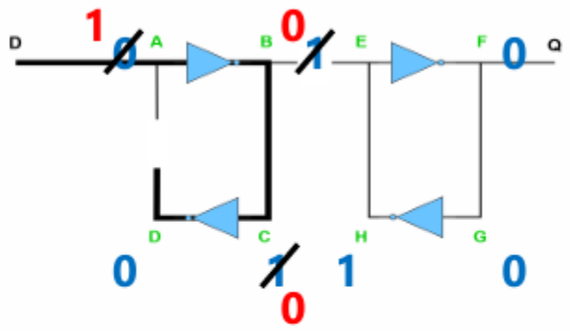
Setup Violation — How Metastability Happens

- Data arrives just before the clock edge and starts overwriting the stored value.
- The edge hits: one inverter loop drives the OLD value, the input drives the NEW value.
- The conflict propagates through every node — the output is neither 0 nor 1.
- **The flop is metastable. After some time one value wins — old or new, we cannot know which.**

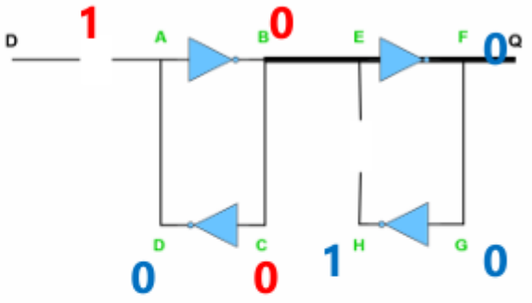
1 To understand how a setup violation happens lets go through this scenario:
Lets assume the FF was storing a logic zero (0)



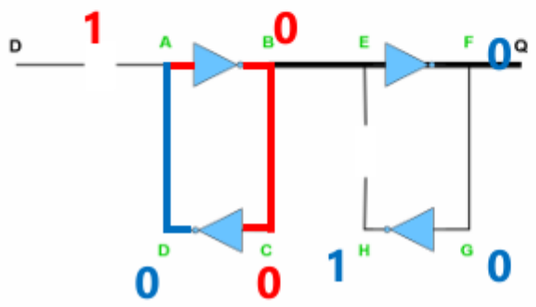
2 Now a **new data** arrives at the D input ,that is logic one (1), and starts overwriting the previous stored value



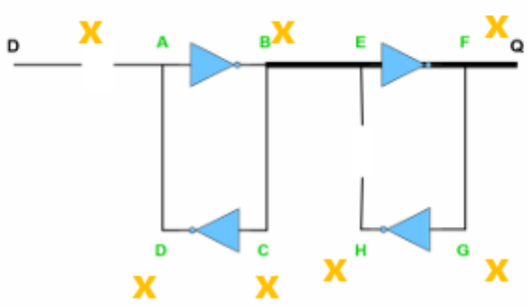
3 The clock edge arrives before the new data have time to overwrite node **D**. The transmission gates switch



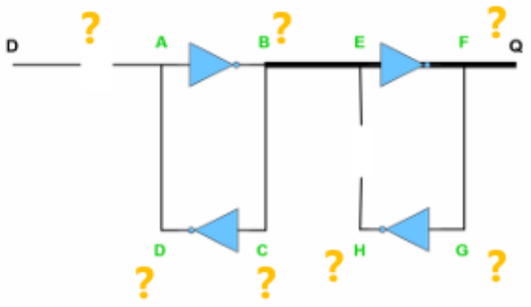
4 The transmission gate between **D** & **A** is now a short circuit, so **D** is trying to force the inverter loop to store the **old data** while **A-B-C** is trying to force the loop to store the **new data**



5 The conflict between the two electrical values will propagate to all the nodes in the FF and the output won't be a 0 or 1. The FF is said to be in a metastable state



6 After some time one of the 2 values will overcome the other and the FF will leave the metastable state. The final state could be the **old data** or the **new data**

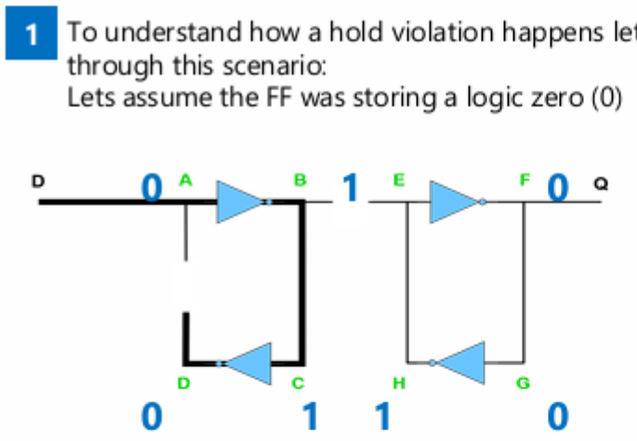


Metastability

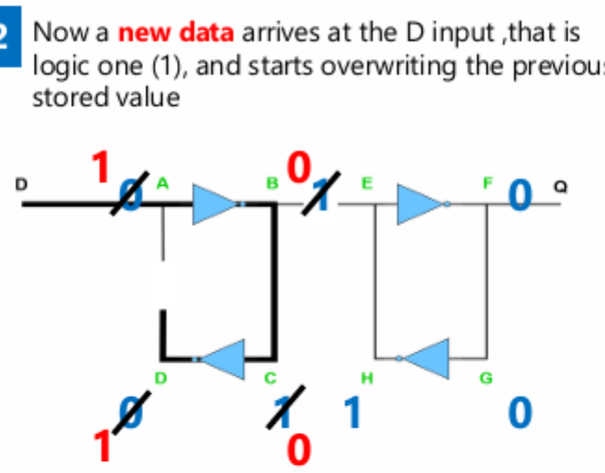
Hold Violation — Same Result

- The edge arrives; the red gates start opening while the green ones close.
- A NEWER data value arrives before the gates finish switching.
- Again two electrical values fight through the inverter loops — all nodes go invalid.
- **Same metastable state, different cause. That is why setup AND hold must both be honored around every edge.**

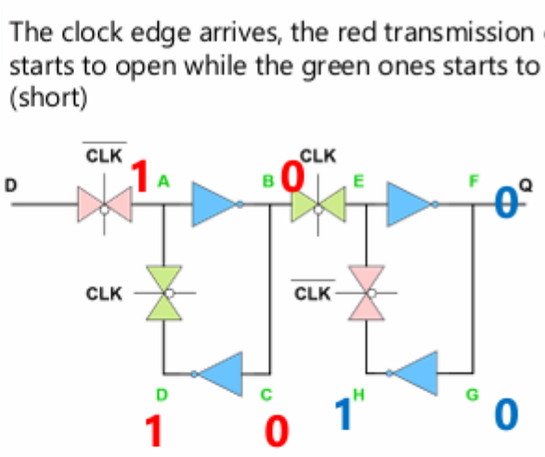
1 To understand how a hold violation happens lets go through this scenario:
Lets assume the FF was storing a logic zero (0)



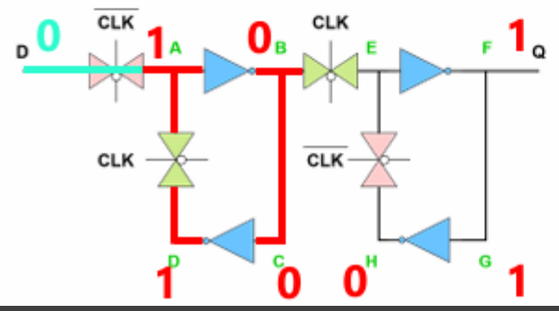
2 Now a **new data** arrives at the D input ,that is logic one (1), and starts overwriting the previous stored value



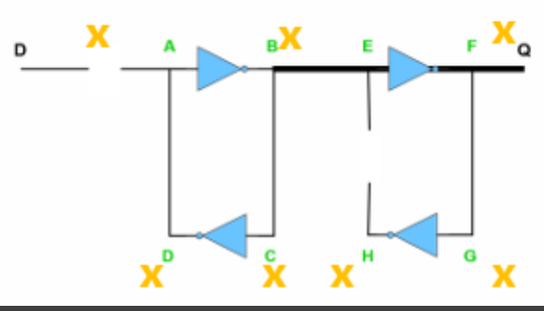
3 The clock edge arrives, the red transmission gates starts to open while the green ones starts to close (short)



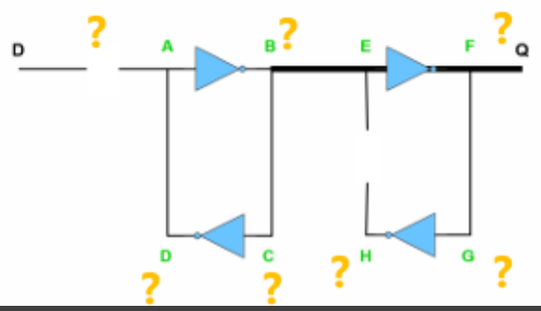
4 Before the red gates completely open, a **newer data** arrives at the D pin, The signal at the D pin is trying to force the inverter loops to store the **newer data**, the nodes **A-B-C-D** are trying to force it to store the **new data**



5 The conflict between the two electrical values will propagate to all the nodes in the FF and the output won't be a 0 or 1. The FF is said to be in a metastable state



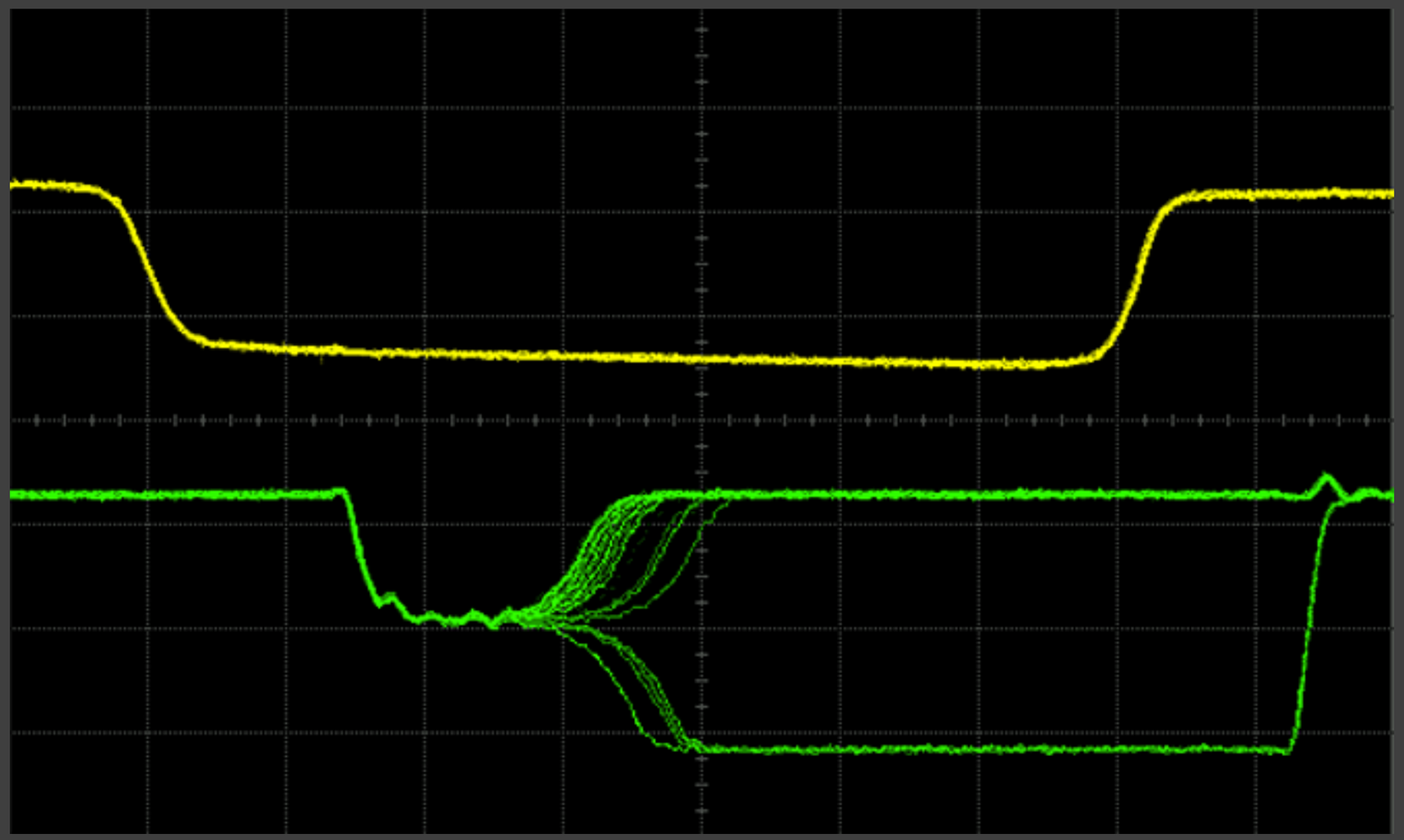
6 After some time one of the 2 values will overcome the other and the FF will leave the metastable state.



Metastability

Metastability in Time

- The metastable voltage hovers between logic levels, then decays to a valid 0 or 1.
- The settling time is random — sometimes one gate delay, sometimes many.
- **Key idea: give the metastable value TIME and ISOLATION to settle before anything else reads it.**



Metastability

Why STA Cannot Help

- STA works when we KNOW when data arrives relative to the clock edge.
- Same clock source: launch/capture edges are related by calculable skew — STA works.
- Divided clocks: divider delays are known — STA still works.
- **Two independent PLLs: phase difference unknown — STA is impossible, we need synchronizers.**

Metastability

The Five CDC Concerns

- **Data corruption:** metastability settles to 0 or 1 regardless of the true input — **missing value**.
- **Data incoherence:** different blocks see different values of the same crossing signal — each **FF** settles the metastability with a different value.
- **Data loss:** fast-to-slow crossings — samples skipped because the slow domain never sees them.
- **Data duplication:** slow-to-fast crossings — sample the same signal at the receiver multiple times.
- **Chip burning:** during metastability both PMOS and NMOS are ON — huge short-circuit current.

- **Each technique in this deck fixes some of these — track them as we go.**

MTBF

MTBF

You Cannot Avoid Metastability

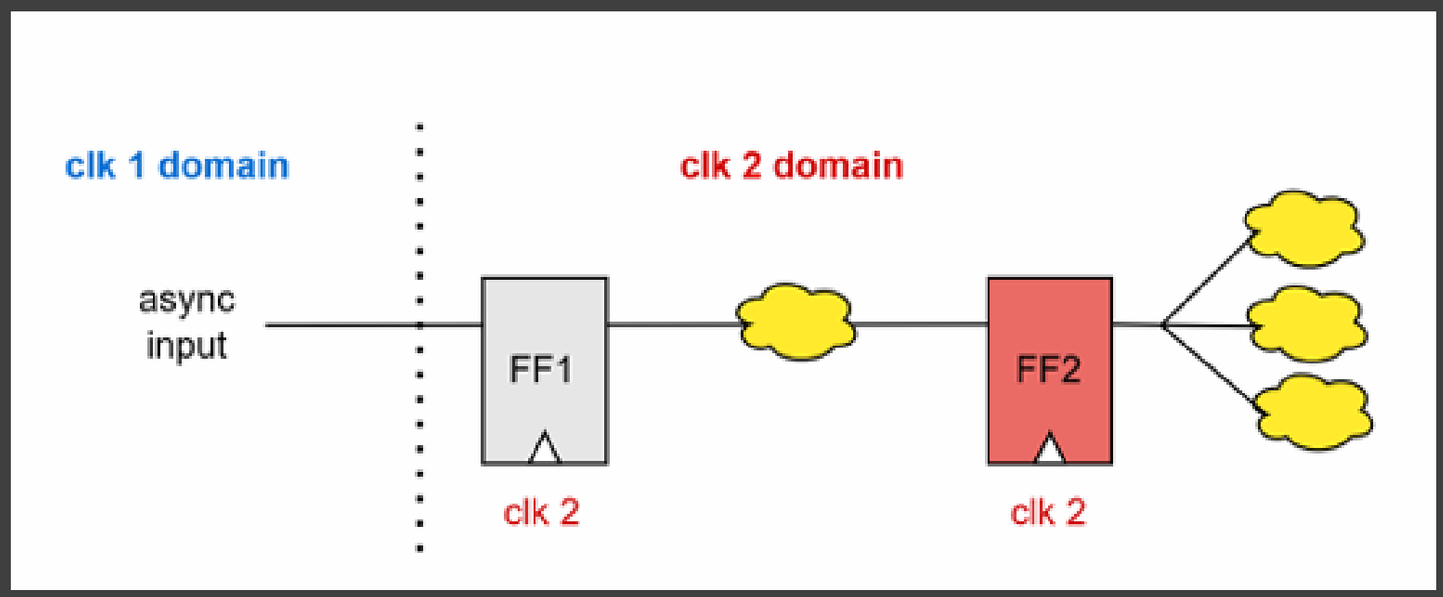
- Step 1 — accept it: the first flop to sample an async signal WILL go metastable sometimes.
- Step 2 — isolate it: make sure the metastable value never reaches FF2 and the rest of the chip.
- Step 3 — quantify it:
- **MTBF — Mean Time Between Failures: the average time between metastability failures escaping the synchronizer.**
- Our job: make MTBF longer than the product lifetime.

MTBF

MTBF Derivation (1) — How Often FF1 Goes Metastable

$$f_{meta} = \alpha * f_{clk_1} * f_{clk_2} * T_o$$

- $f_{clk_1} * \alpha$**:** toggle frequency of the async input (α = activity factor; α >1 means glitches).
- f_{clk_2}**:** more capture edges per second = more chances to sample inside the window.
- $T_o = T_{setup} + T_{hold}$**:** the forbidden window
- *Higher frequencies and glitchier signals shrink the MTBF linearly.*



MTBF

MTBF Derivation (2) — Will It Reach FF2?

$$P_u = e^{-\frac{t_w}{t_{tech}}}$$

- P_u: probability the value is STILL metastable after the settle window t_w.
- t_w : the positive setup slack from FF1 to FF2 (Tclk2 - Tcq - Tcomb - Tsetup).
- t_{tech} : technology time constant — how fast the loop resolves (RC + gain).

$$MTBF = \frac{1}{frequency\ of\ failure}$$

$$= \frac{1}{P_u * f_{meta}} = \frac{e^{\frac{t_w}{t_{tech}}}}{\alpha * F_{clk_1} * F_{clk_2} * T_o}$$

- Levers: fewer toggles (a), smaller To (better cells), bigger tw (less logic between the sync flops), more stages.

MTBF

Adding Synchronizer Stages

- So, the frequency of failure where FF1 and FF2 both would generate metastable state is $F_{meta} * P_u$.

- If we added extra flop after FF2 the frequency of failure will decrease and so the MTBF will increase.

$$frequency\ of\ failure_1 = \left(\alpha * F_{clk_1} * F_{clk_2} * T_o * e^{\frac{-t_{u_1}}{t_{tech_1}}}\right)$$

$$frequency\ of\ failure_2 = frequency\ of\ failure_1 * F_{clK_2} * T_o * e^{\frac{-t_{u_2}}{t_{tech_2}}}$$

- **2 stages are fine for low frequency designs; high-speed designs need 3+.**
- **Cost:** one more cycle of latency per stage.

MTBF

How to Increase the MTBF

- Add synchronizer stages (2 for slow domains, 3+ for fast ones).
- No combinational logic in front of the synchronizers (a <= 1, no glitches).
- Register the signal in the SENDING domain before the crossing.
- Place the synchronizer flops close together in PNR — minimal wire delay, bigger tw.
- Use dedicated synchronizer cells from the library (optimized Tsetup/Thold/tau).

MTBF

What the Synchronizer Solves — Concerns Tracker

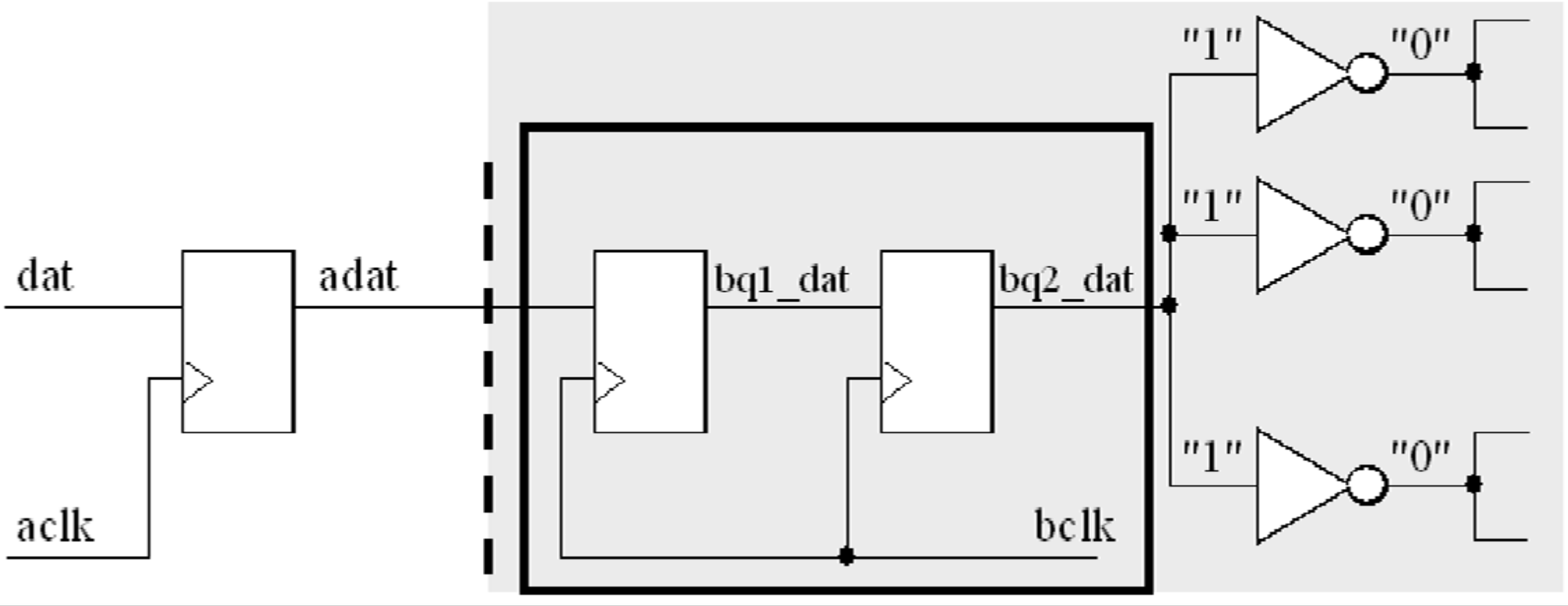
Concern	Status after plain 2-FF synchronizer
Data corruption	NOT fixed — value can settle wrong; system must tolerate it
Data incoherence	Fixed — settles once, then fans out the same value
Data loss	NOT fixed — narrow pulses can still be missed
Data duplication	NOT fixed — wide pulses sampled twice
Chip burning	Fixed — metastability isolated and rare

Synchronizers

Synchronizers

Two Flip-Flop Synchronizer

- The first flop takes the metastability hit; the second gives it a full clock cycle to settle.
- FF2 output is a clean 0 or 1 — one cycle late, and possibly the **WRONG** value (corruption is not fixed).
- Never put logic between the two flops — $\alpha \geq 1$ and never fan out from the first flop.



Synchronizers

Synchronizer — Code

- Both flops in one module; the pair shifts d into the rx domain.
- Reuse ONE instance per crossing — never build a second sync for the same signal (divergence rule).

CDC - 00_two_flop_synchronizer.v

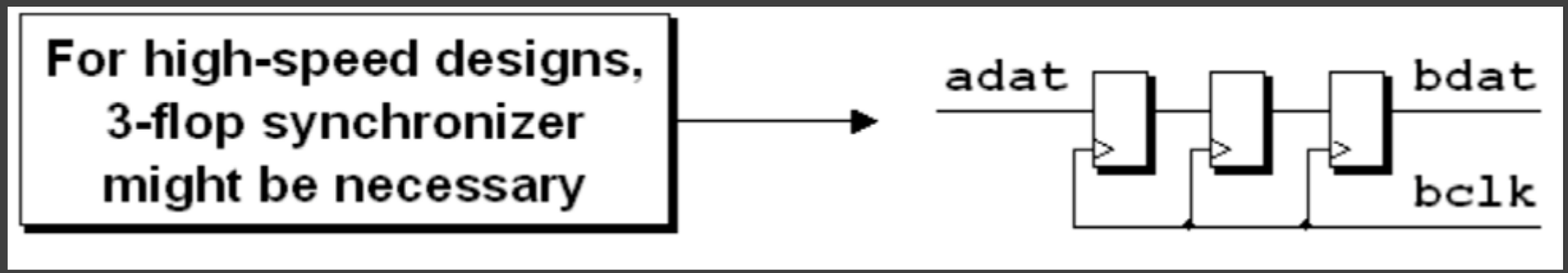
```
// 2-flop synchronizer: the basic CDC building block.
// * FF1 takes the metastability hit, FF2 gives it one rx_clk cycle to settle
// * Rules: no logic between the flops, no fanout from FF1,
//   ONE instance per signal (never two syncs for the same crossing)
module two_flop_synchronizer (
    output logic q,        // clean, synchronized signal in the rx domain
    input  logic d,        // async input - register it in the tx domain first!
    input  logic clk2,     // receiving (destination) clock
    input  logic rst_n
);
    logic q1;
    always_ff @(posedge clk2 or negedge rst_n)
        if (!rst_n) {q, q1} <= '0;
        else        {q, q1} <= {q1, d};
endmodule

```

Synchronizers

Three Flops and Synchronizer Cells

- High-speed designs: the one-cycle settle window is too short — add a third stage.
- Cell libraries provide dedicated synchronizer cells with better t_{tech} / setup / hold.

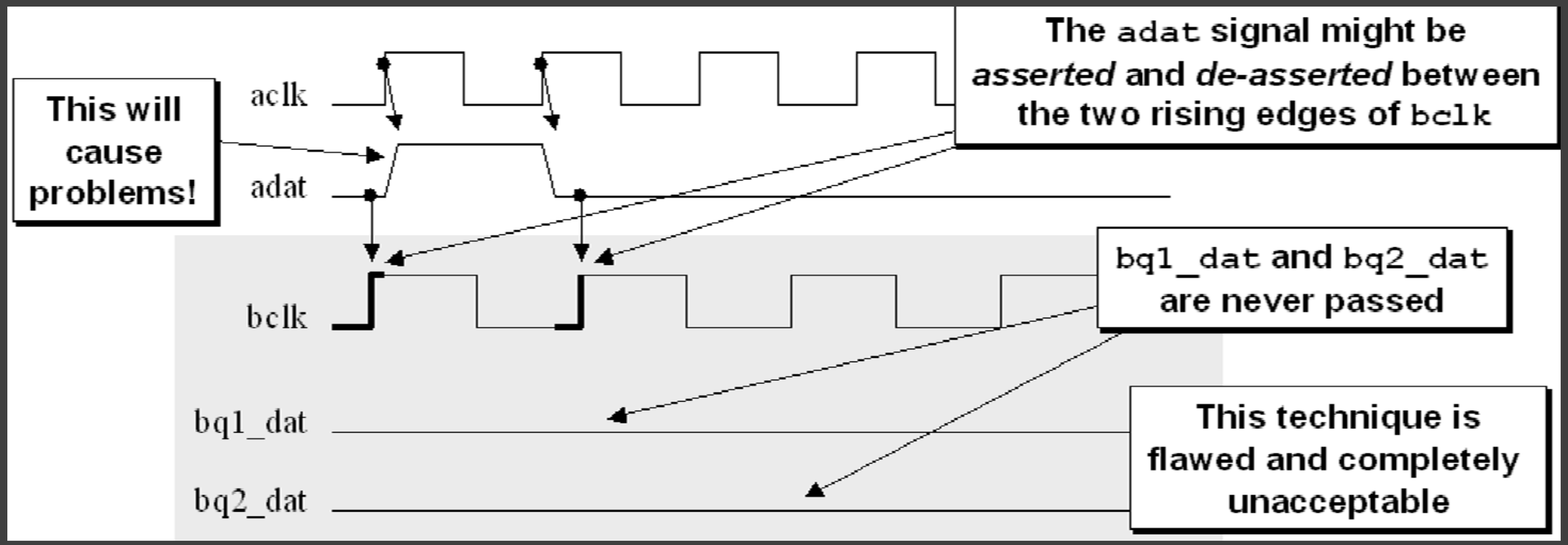


Required Pulse Width

Required Pulse Width

Fast to Slow: Pulses Get Missed

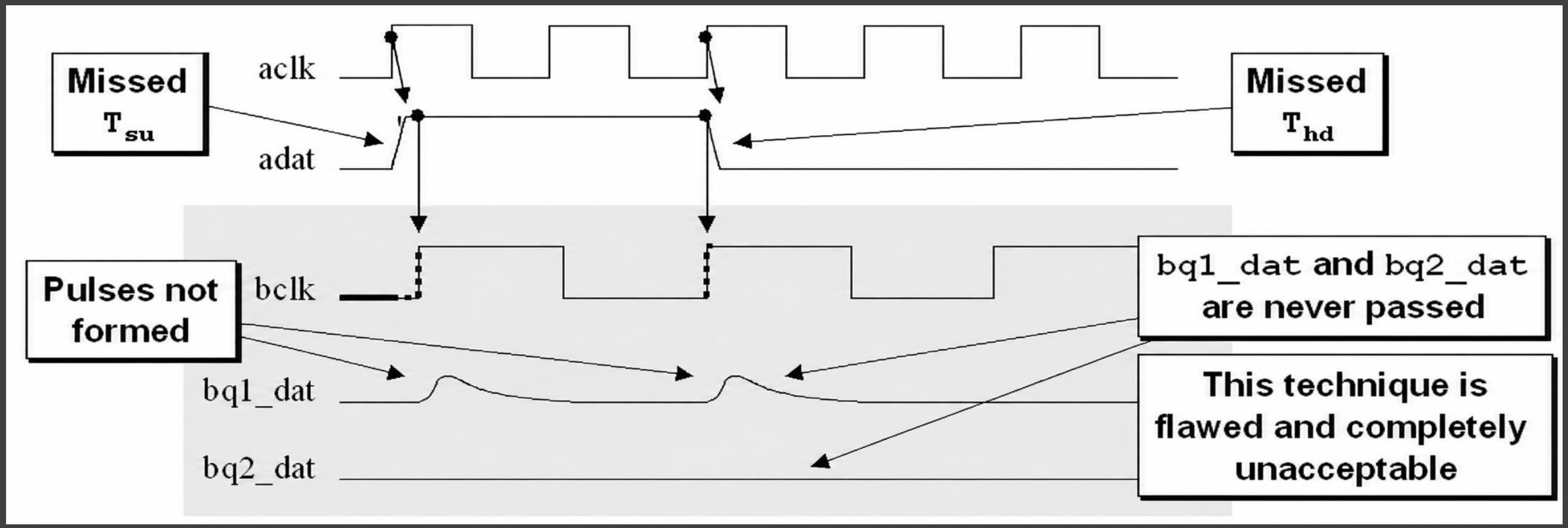
- A one-cycle pulse from a fast clock can fall entirely between two slow clock edges — never sampled.



Required Pulse Width

Fast to Slow: Pulses Get Missed

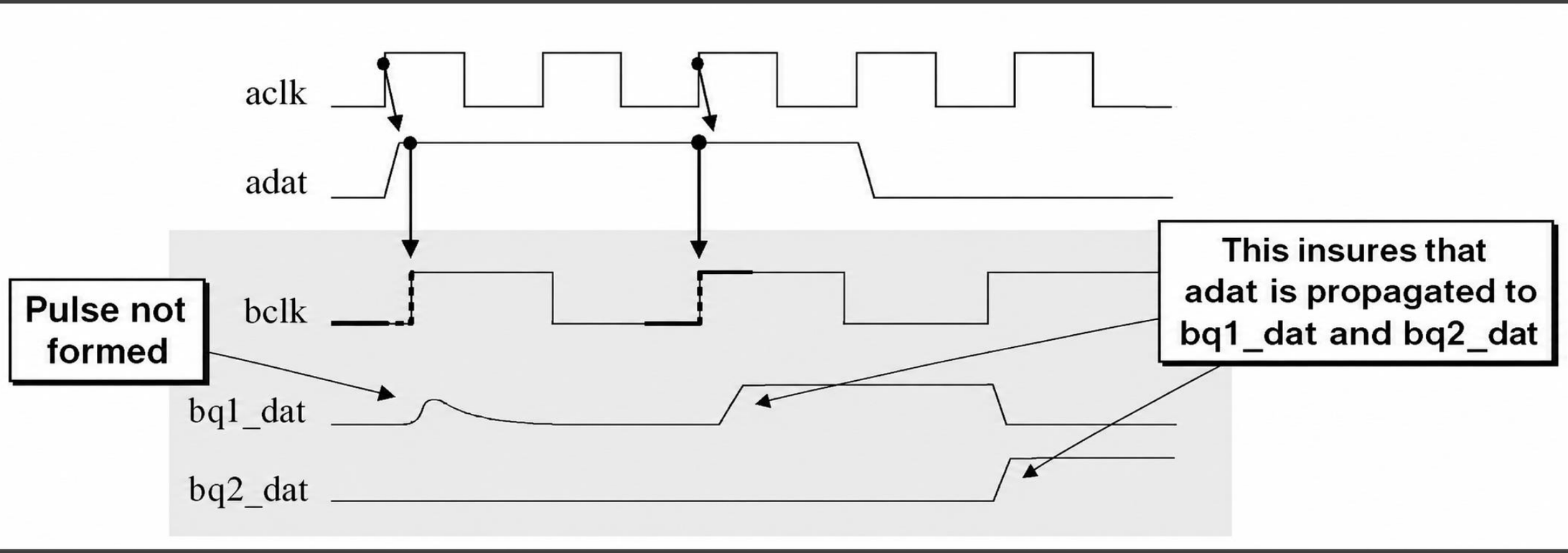
- Trivial solution is to make the pulse width equals to one slow clock period so it can always be samples; However, the pulse can be close to the two rising clock edges causing metastability for both.



Required Pulse Width

The Width Requirement — open loop solution

- Minimum width = Tclk2 + Tsetup + Thold — guarantees some capture edge lands fully inside the pulse.
- **Rule of thumb:** hold the data stable for at least 1.5 x Tclk2.
- This will lead the data to be **correctly** sent but it can be sampled twice or more due to the long pulse width.



Required Pulse Width

Worked Examples

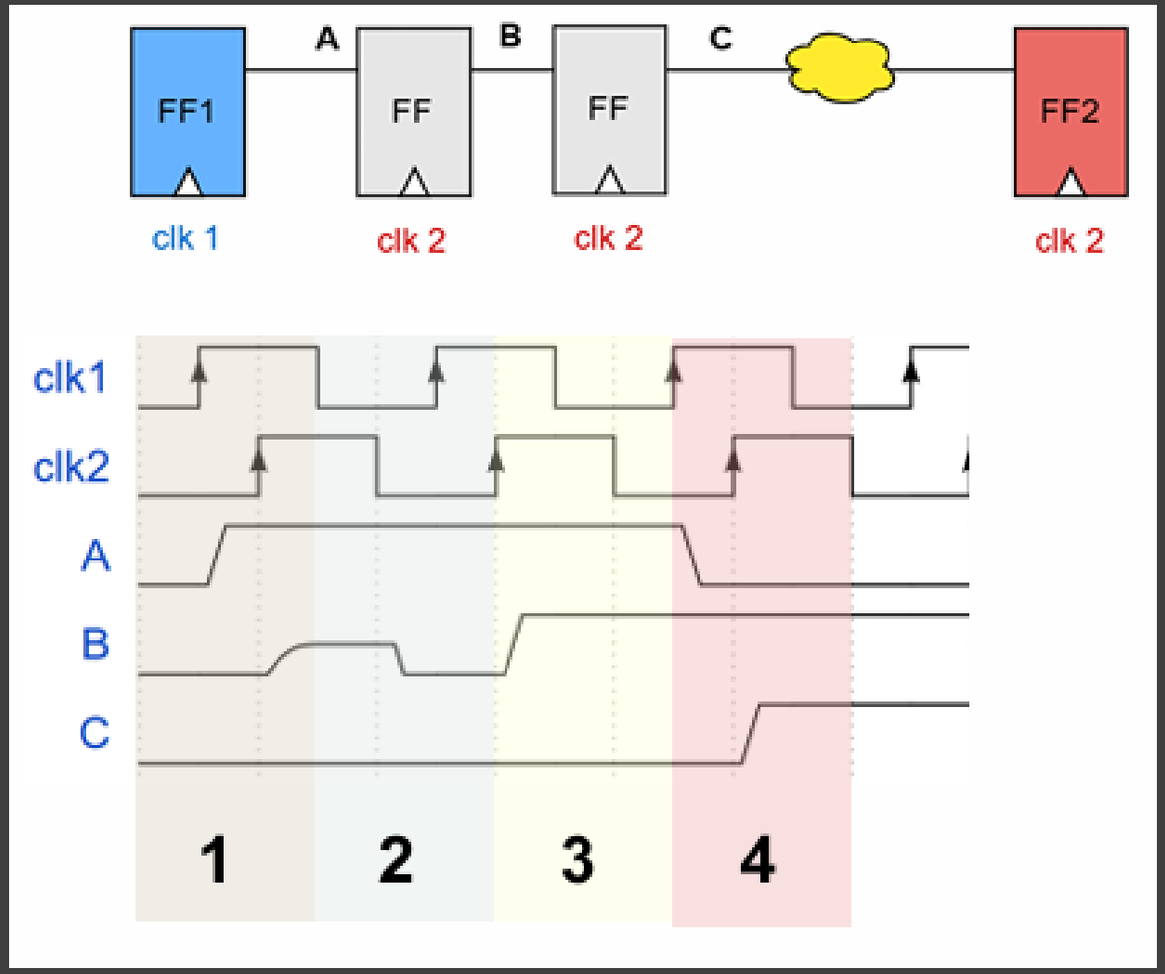
- **Fast to slow:** fclk1 = 225 MHz, fclk2 = 150 MHz.
  - needed width = 1.5 x 6.6ns = 9.9ns -> 9.9 / 4.4 = 2.25 -> hold for 3 clk1 cycles.
- **Slow to fast:** fclk1 = 100 MHz, fclk2 = 120 MHz.
  - needed width = 1.5 x 8.3ns = 12.45ns -> 12.45 / 10 = 1.245 -> hold for 2 clk1 cycles.

- **Synchronizers make the crossing SAFE; the width makes it CORRECT.**

Required Pulse Width

Varying Settling Time

- If the first sync flop resolves the metastable value correctly, the data arrives one cycle **EARLIER** at the logic — C will be one at stage 3 not 4.
- The crossing delay is therefore +/-(1 cycle) — **unavoidable and random**.
- **The design must work with either arrival time.**
- Advanced CDC tools inject varying delays into CDC paths and verify both cases.

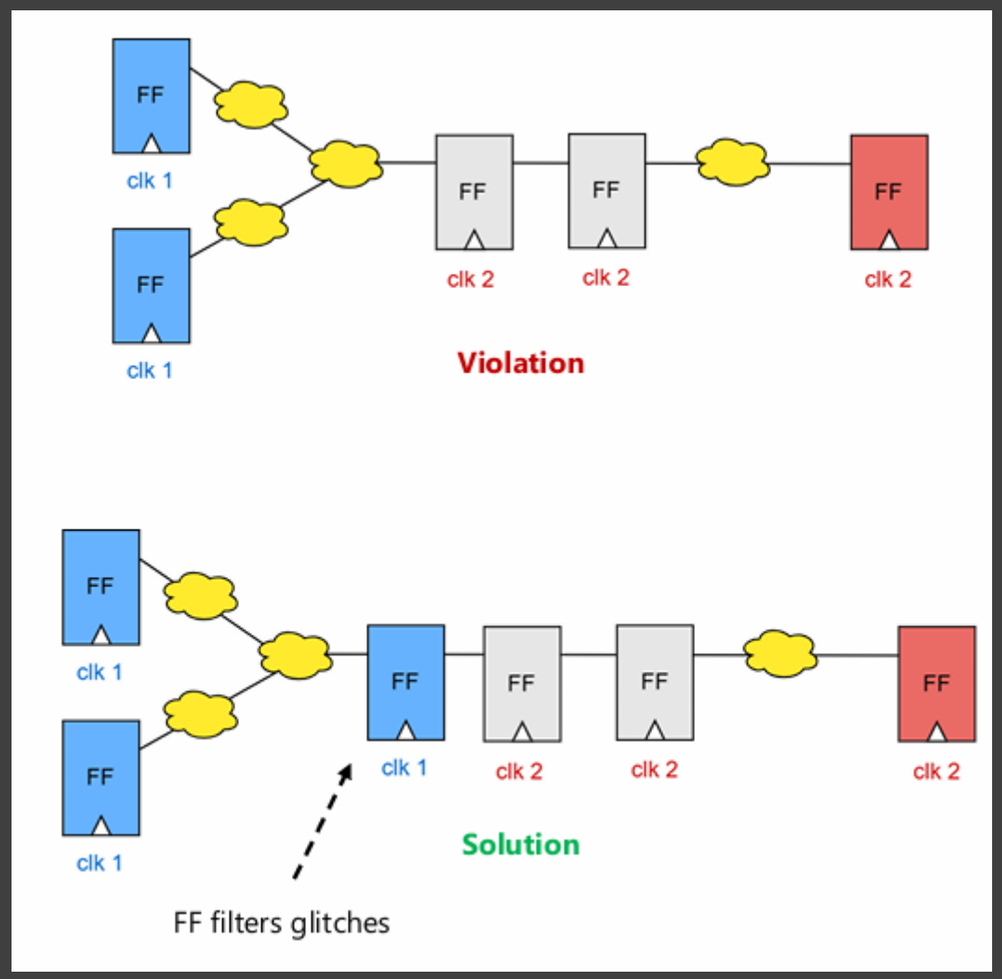


CDC Rules

CDC Rules

Rule 1 — No Combinational Logic in Front of Syncs

- Logic converging in the sending domain generates glitches — every glitch is a potential metastability event (activity factor $\alpha > 1$).
- **Fix: register the converged signal in the sending domain, then let the clean FF output drive the synchronizer.**



CDC - 13_comb_fanin_bad.v

```
// VIOLATION vs FIX: combinational logic in front of a CDC synchronizer.
// The AND output glitches; every glitch is a metastability candidate
// (activity factor a > 1, MTBF drops). CDC tools flag this as critical.
module comb_fanin_bad (
    output logic q_sync,
    input  logic a, b,        // tx-domain signals
    input  logic clk2, rst_n
);
    wire comb = a & b;        // GLITCHY - must not feed a synchronizer
    logic q1;
    always_ff @(posedge clk2 or negedge rst_n)
        if (!rst_n) {q_sync, q1} <= '0;
        else        {q_sync, q1} <= {q1, comb};   // <-- violation
endmodule

// GOOD: register the converged signal in the SENDING domain first;
// the FF filters the glitches, then the clean output crosses.
module comb_fanin_good (
    output logic q_sync,
    input  logic a, b,
    input  logic clk1, clk2, rst_n
);
    logic comb_q;
    always_ff @(posedge clk1 or negedge rst_n)
        if (!rst_n) comb_q <= '0;
        else        comb_q <= a & b;              // glitch filter

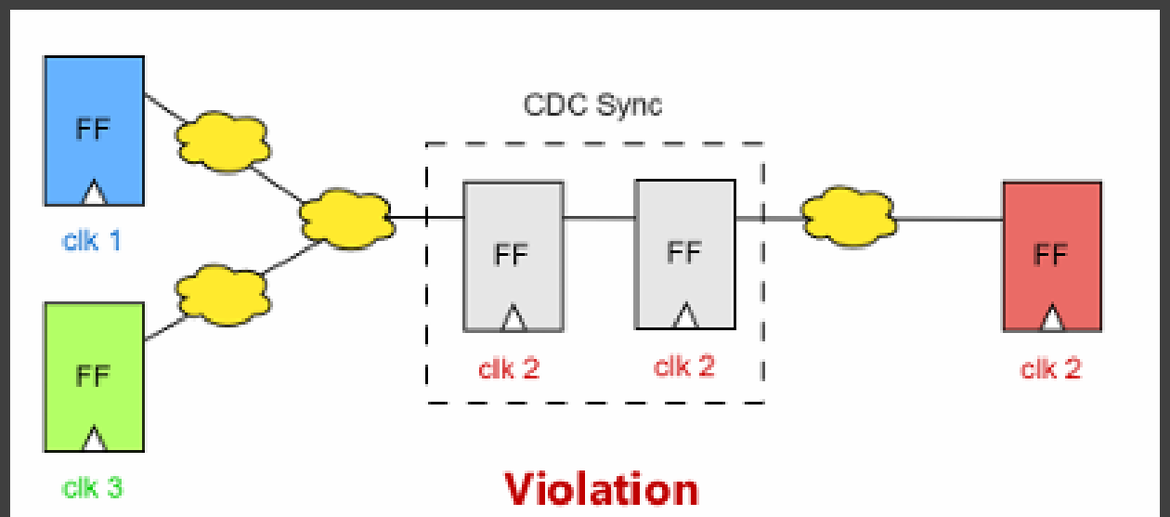
    logic q1;
    always_ff @(posedge clk2 or negedge rst_n)
        if (!rst_n) {q_sync, q1} <= '0;
        else        {q_sync, q1} <= {q1, comb_q}; // clean crossing
endmodule

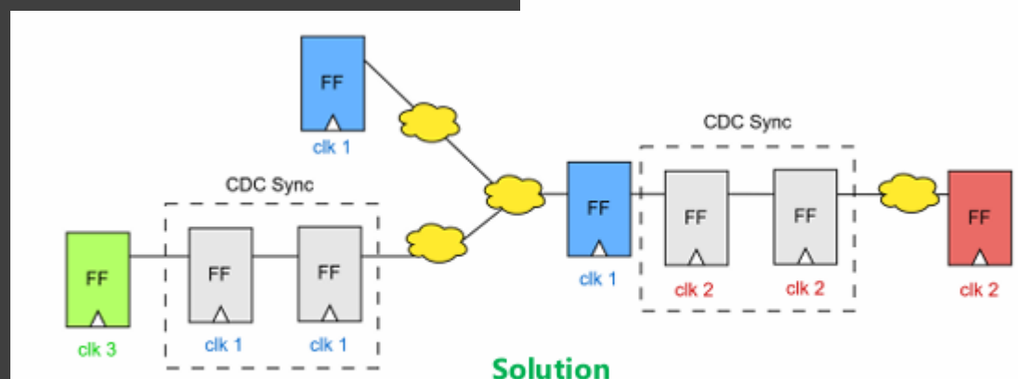
```

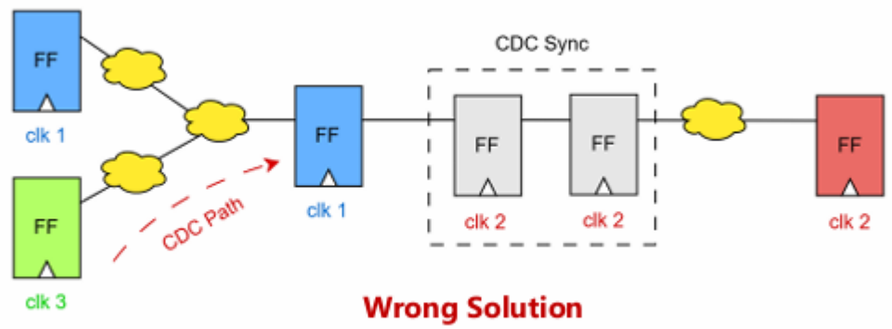
CDC Rules

Rule 2 — Multi-Clock Fan-In

- Signals from MULTIPLE clock domains converging into one synchronizer — more glitches, and CDC tools cannot even classify the crossing.
- **Wrong fix:** adding a filtering FF — that just creates a new async crossing.
- **Fix: synchronize one input into the other domain first, compute in ONE domain, then cross once.**



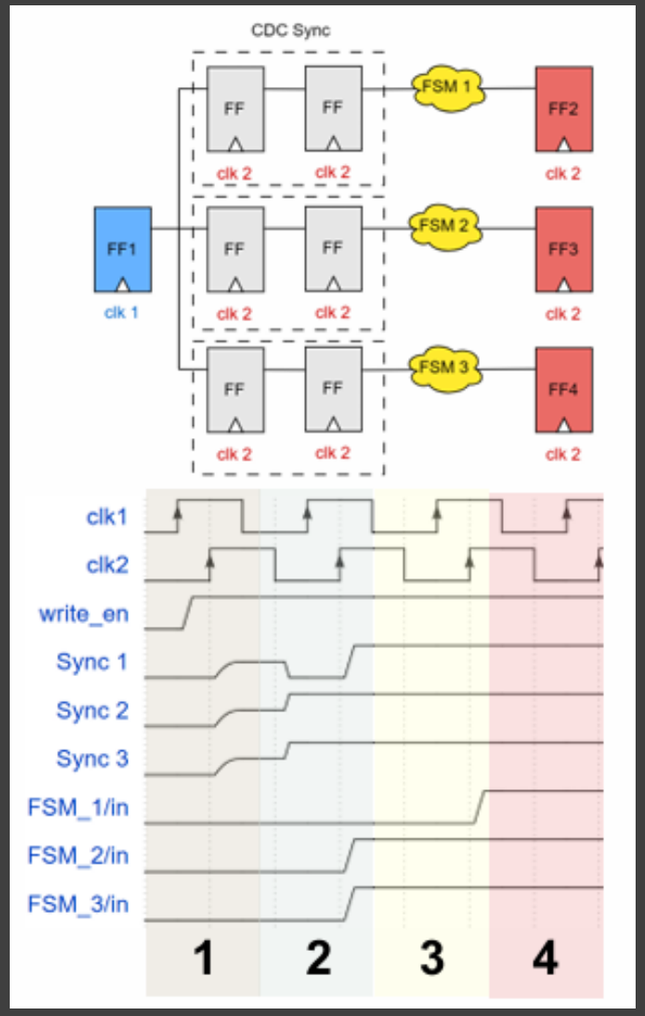


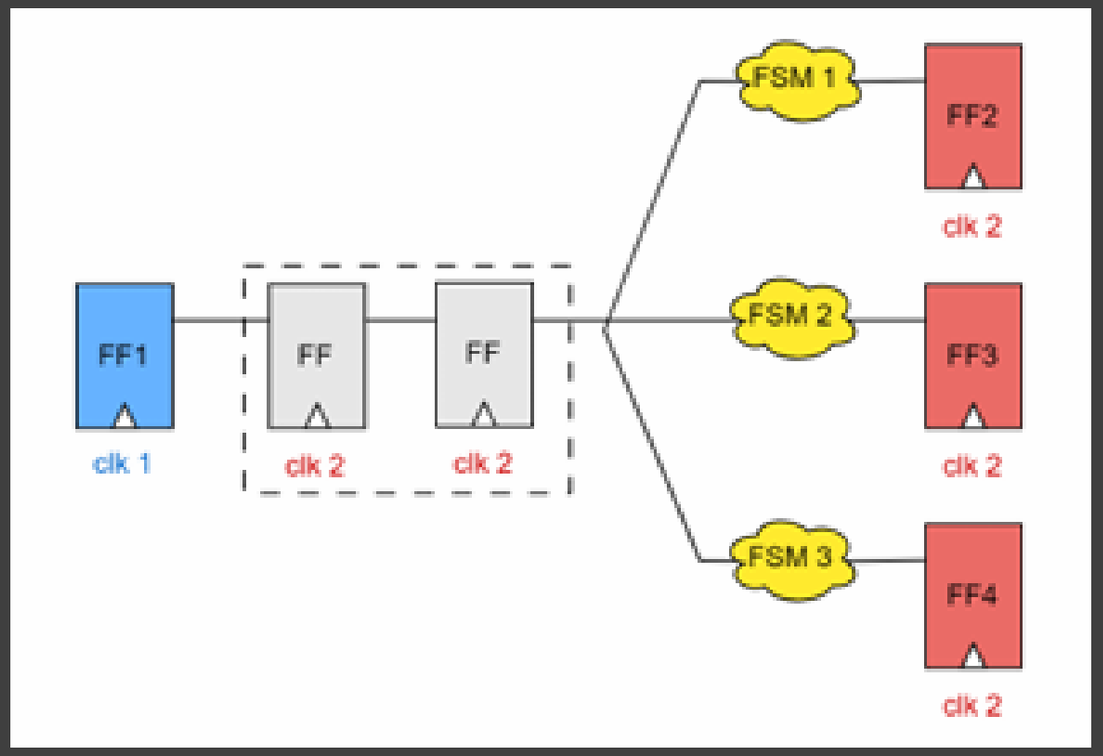


CDC Rules

Rule 3 — Divergence: One Signal, One Synchronizer

- The same async signal sent through THREE parallel synchronizers: each may settle differently — FSMs in the rx domain disagree.
- Happens in real teams: two engineers each add their own sync for a shared global signal.
- **Fix: ONE synchronizer per crossing (a dedicated CDC module), fan out only in the receiving domain.**

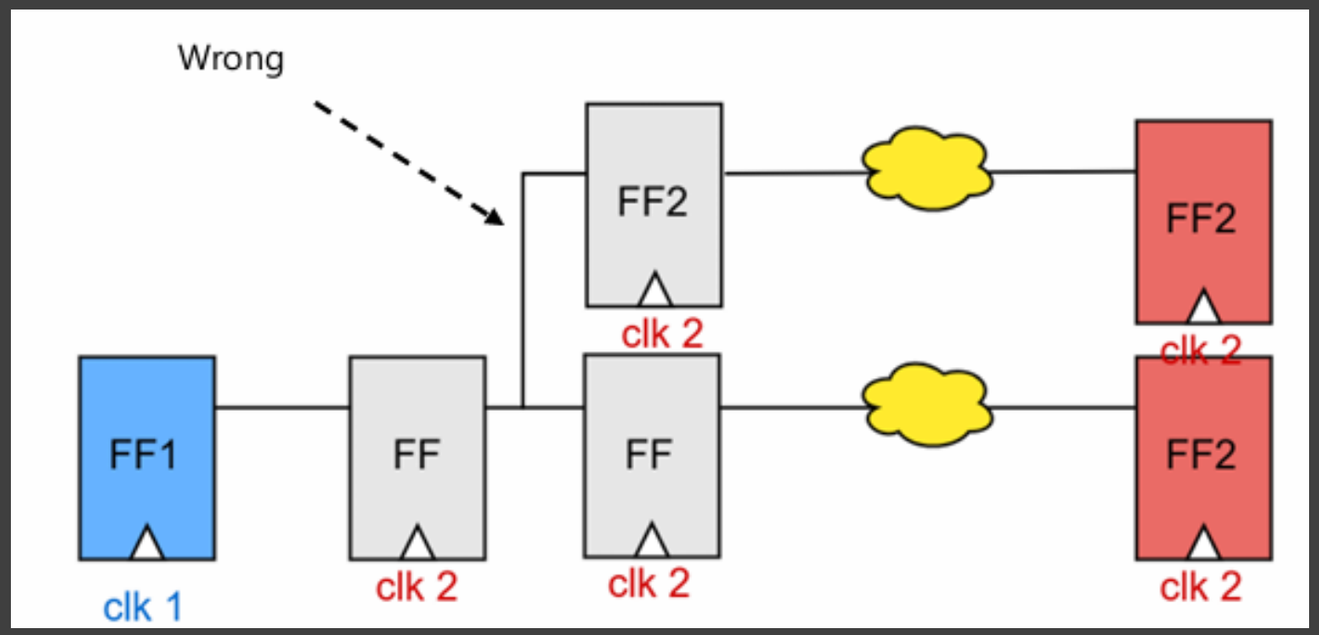




CDC Rules

Rule 4 — Never Tap Between Sync Stages

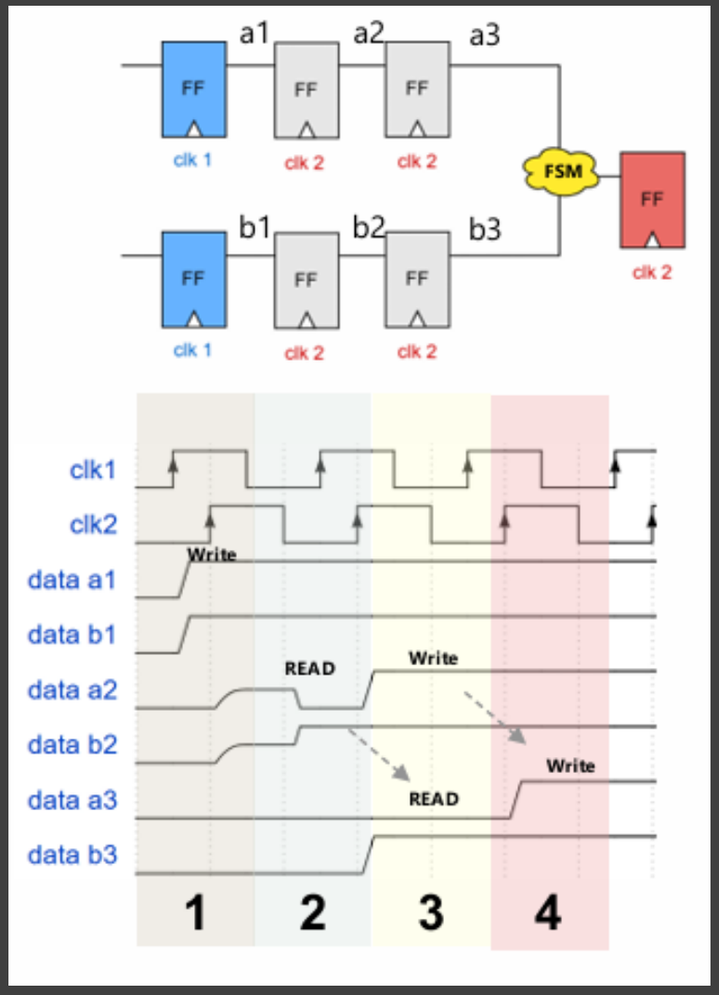
- Reading the signal BETWEEN the synchronizer flops fans out a possibly-metastable value.
- Only the final synchronizer output may leave the CDC module.



CDC Rules

Rule 5 — Multi-bit Reconvergence

- Two bits of one bus cross through separate synchronizers and re-converge in the rx domain.
- They can settle one cycle apart: 2'b11 (WRITE) arrives as 2'b10 (READ) for a cycle — a legal but WRONG value.
- **Fixes:** converge in the sending domain, gray-code the value, or pass the bus as a group (MUX/MCP/FIFO).



CDC - 09_unsafe_multi_bit_crossing.v

```
1  // UNSAFE: a 2-bit FSM state crossing through TWO independent synchronize
   rs.
2  // The two bits can settle one cycle apart: state 2'b11 (WRITE) can be
3  // received as 2'b10 (READ) for a cycle - a legal but WRONG state.
4  // This is the Amr deck part-2/part-4 reconvergence walk-through in RTL.
5  // FIXES: converge to 1 bit in the tx domain, gray-code the value,
6  // or pass the bus as a group (MUX / MCP / FIFO).
7  module unsafe_multi_bit_crossing (
8      output logic [1:0] state_rx,
9      input  logic [1:0] state_tx,   // e.g. IDLE=2'b00, WRITE=2'b11, READ=
   2'b10
10     input  logic clk2, rst_n
11 );
12     logic sa1, sa2, sb1, sb2;      // two independent 2-FF synchronizers
13     always_ff @(posedge clk2 or negedge rst_n)
14         if (!rst_n) {sa2, sa1} <= '0;
15         else        {sa2, sa1} <= {sa1, state_tx[1]};   // bit 1
16     always_ff @(posedge clk2 or negedge rst_n)
17         if (!rst_n) {sb2, sb1} <= '0;
18         else        {sb2, sb1} <= {sb1, state_tx[0]};   // bit 0
19     assign state_rx = {sa2, sb2};  // can be a MIXTURE of old and new!
20 endmodule
21
```

Passing Events (1-bit)

Passing Events (1-bit)

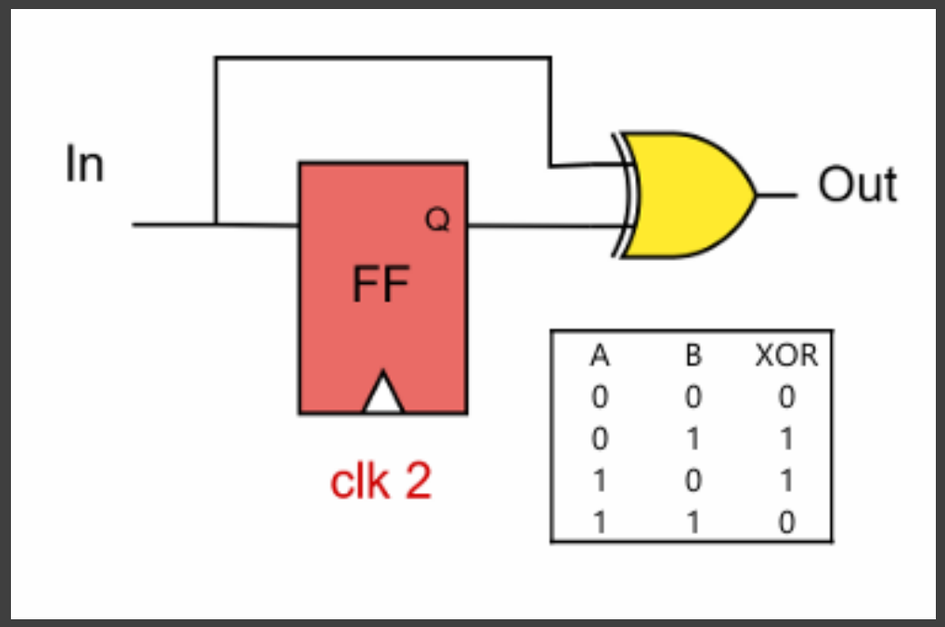
The Data Duplication Problem

- Domain 1 sends an enable to a counter. To be safely captured the pulse must be WIDE (1.5 x Tclk2).
- The wide pulse is then sampled by TWO slow-clock edges — the counter counts twice.
- **We need the rx domain to see exactly ONE cycle per event, no matter how wide the incoming pulse is.**

Passing Events (1-bit)

Pulse Generator

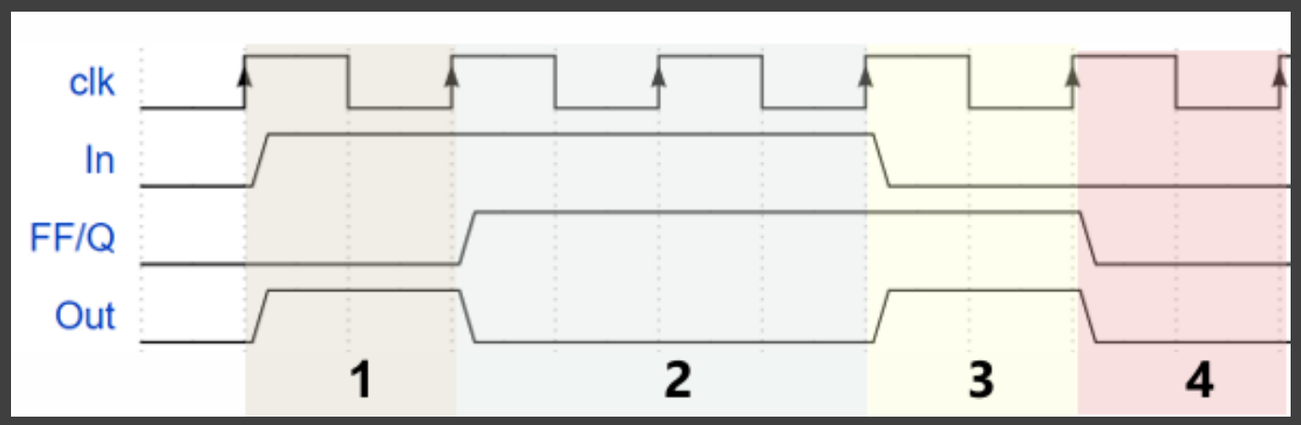
- XOR of the input with its registered version: outputs 1 for exactly one cycle on EVERY toggle (0->1 and 1->0).
- Feed it a **TOGGLING** signal from the tx domain -> a clean 1-cycle event per toggle, immune to duplication.



CDC - 03_pulse_generator.v

```v
// Pulse generator: a 1-cycle pulse on EVERY toggle of d (0->1 AND 1->0).
// XOR of the input with its registered previous value.
// Feed it the synchronized level of a TOGGLING tx signal and you get
// exactly one clean rx event per toggle - fixes data duplication.
module pulse_generator (
    output logic pulse,   // 1-cycle pulse per toggle of d
    output logic q,       // registered copy of d (for chaining)
    input  logic d,
    input  logic clk,     // rx-domain clock
    input  logic rst_n
);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) q <= '0;
        else        q <= d;
    assign pulse = q ^ d;
endmodule

```

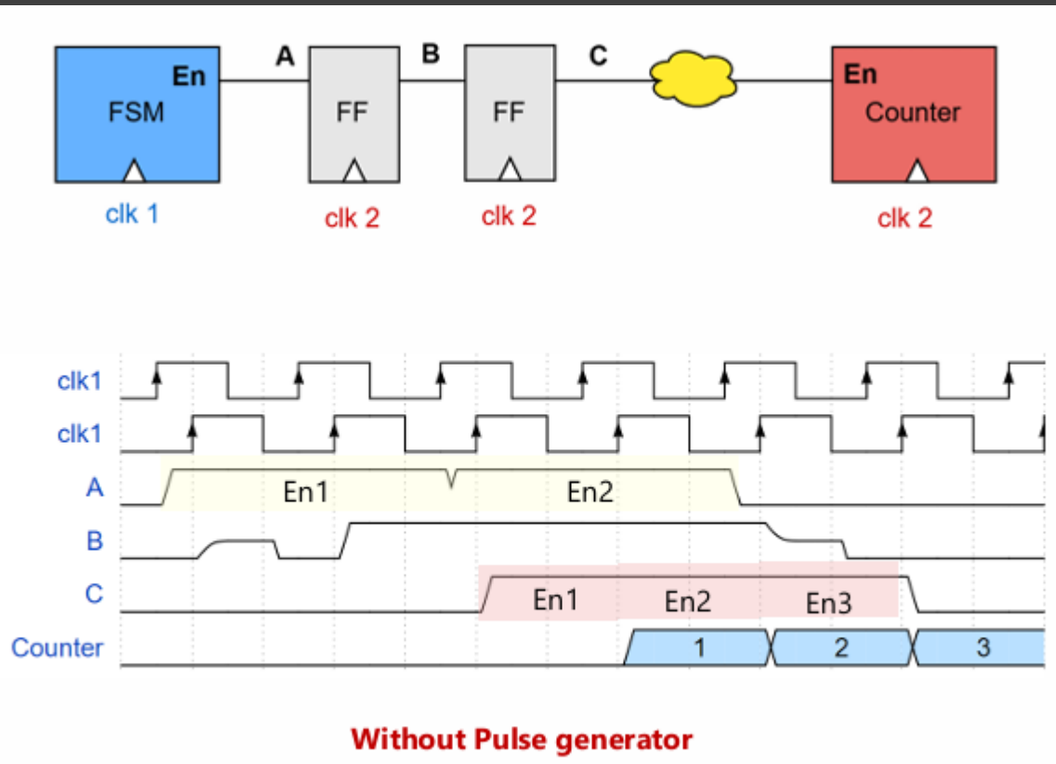


Passing Events (1-bit)

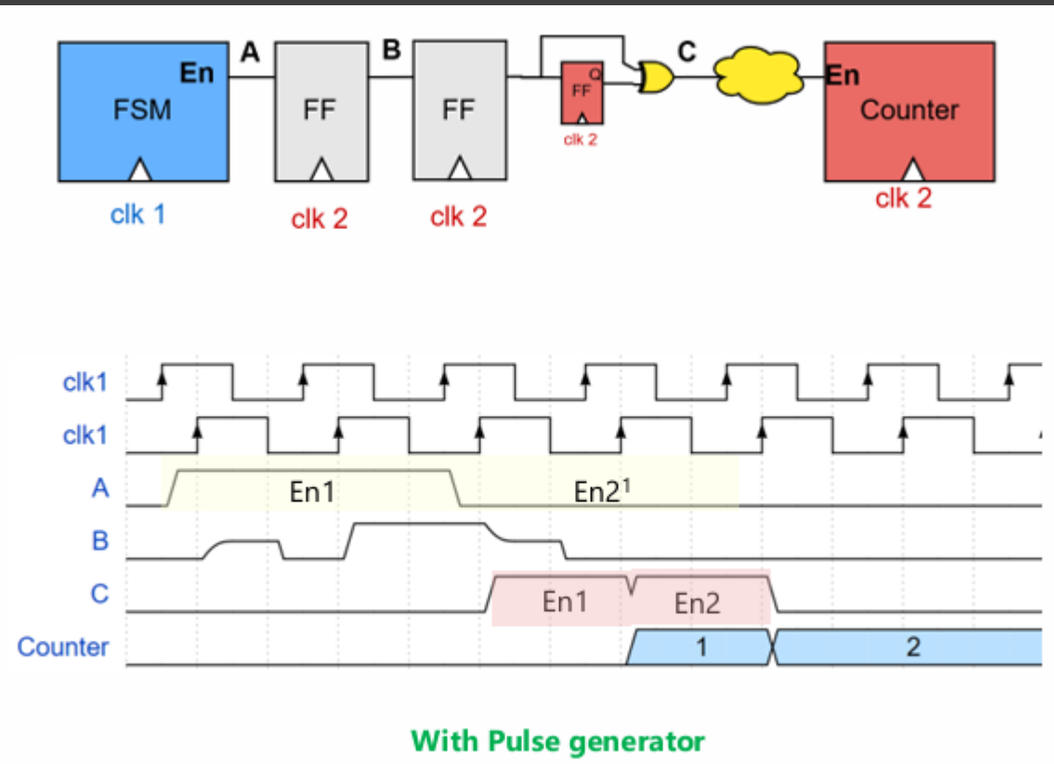
Pulse Generator

- This method needs the input signal to be toggling instead if fixed logic value to be seen as multiple cycles.

- In this example with pulse generator the enable toggled to be seen twice.



Without Pulse generator

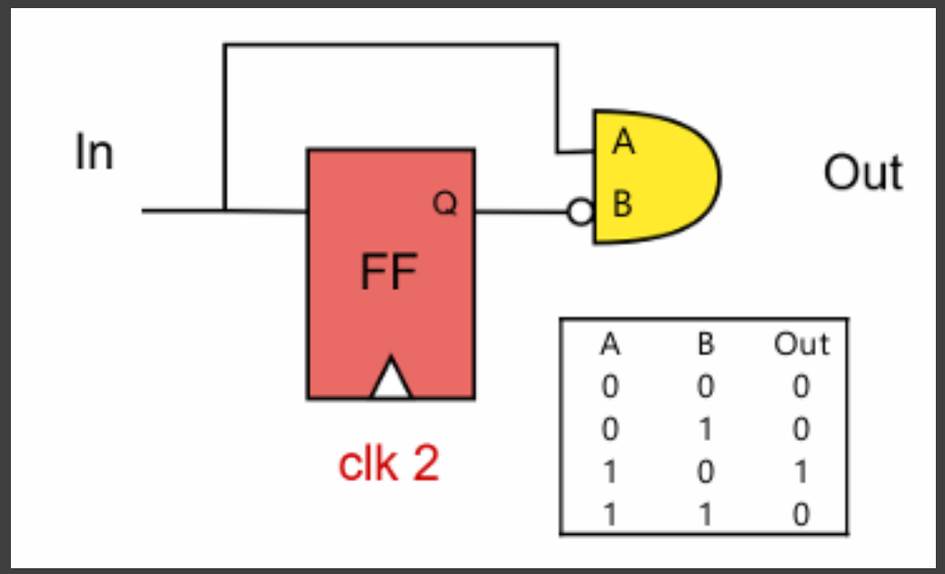


With Pulse generator

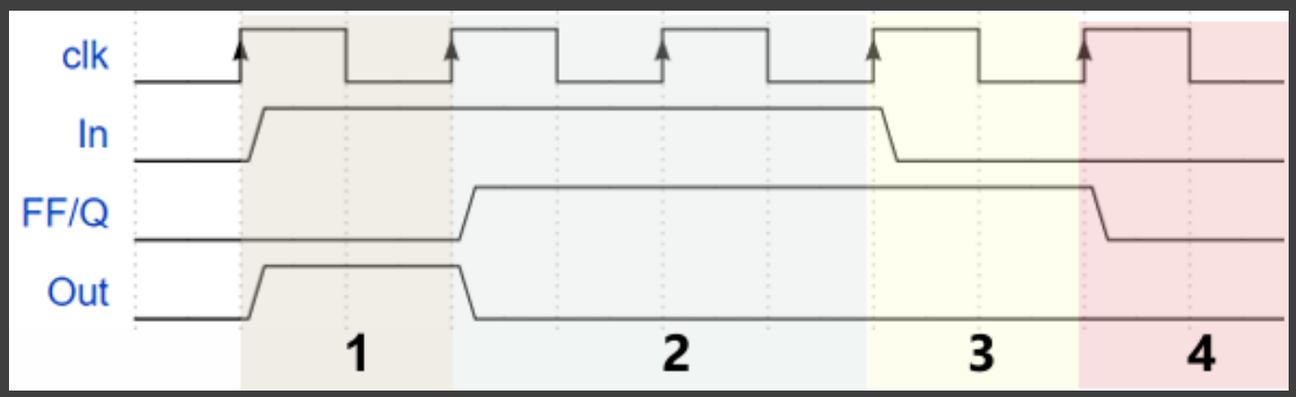
Passing Events (1-bit)

Edge Detector

- AND of the input with the INVERTED registered version: pulses only on 0->1.
- Flip the inputs for falling-edge detection; use NOR/AND combinations for active-low outputs.



A	B	Out
0	0	0
0	1	0
1	0	1
1	1	0



CDC - 04_edge_detector.v

```
// Edge detectors: 1-cycle pulse on ONE edge direction only.
// * rising  : pulse =  d & ~q
// * falling : pulse = ~d &  q
// Active-low output variants: NOR the pair instead of AND.
module edge_detector_rising (
    output logic pulse,   // 1-cycle pulse on every 0 -> 1 of d
    output logic q,
    input  logic d,
    input  logic clk,
    input  logic rst_n
);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) q <= '0;
        else        q <= d;
    assign pulse = d & ~q;
endmodule

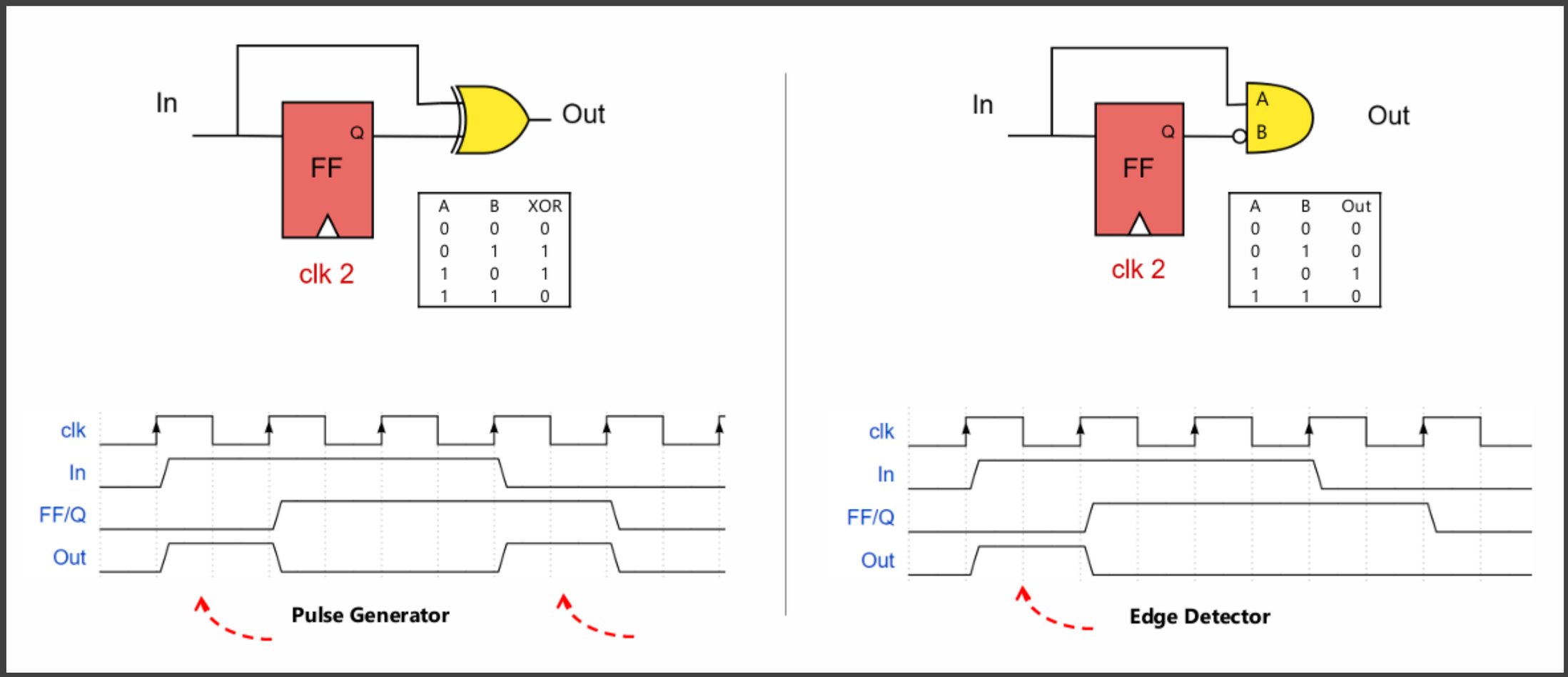
module edge_detector_falling (
    output logic pulse,   // 1-cycle pulse on every 1 -> 0 of d
    output logic q,
    input  logic d,
    input  logic clk,
    input  logic rst_n
);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) q <= '0;
        else        q <= d;
    assign pulse = ~d & q;
endmodule

```

Passing Events (1-bit)

Pulse Generator vs Edge Detector

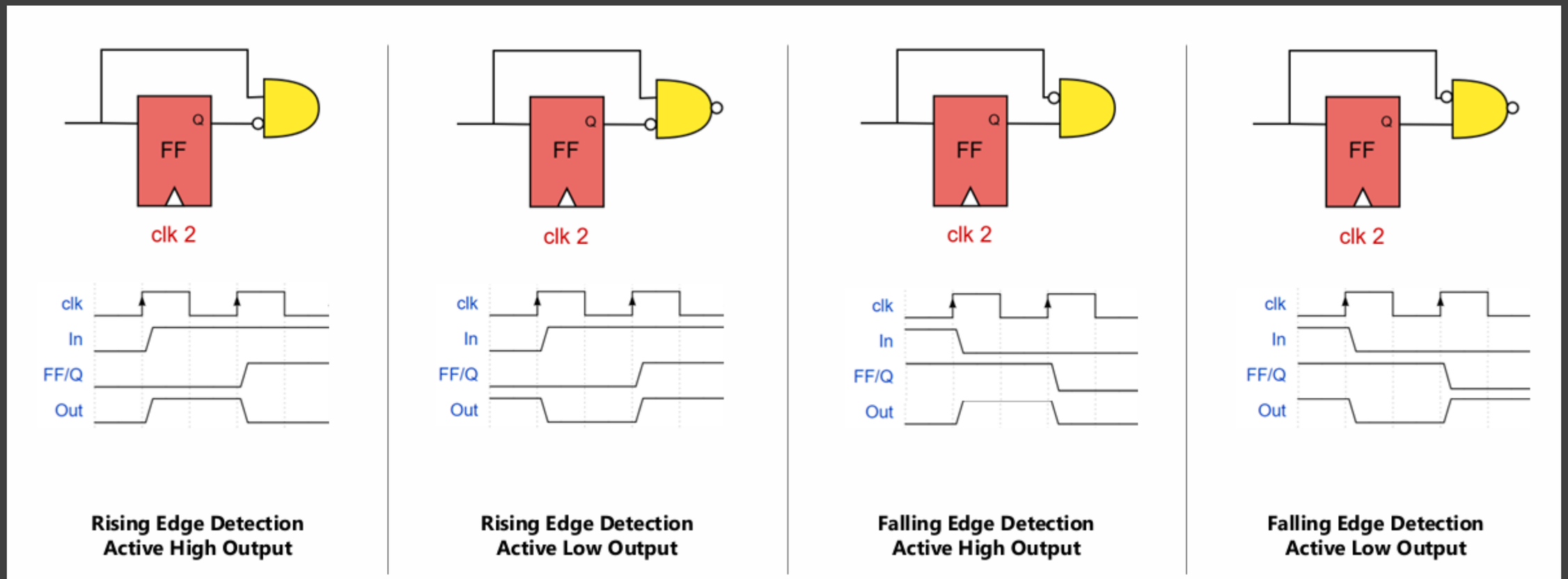
- Pulse generator = XOR: fires on BOTH edges. Edge detector = AND: fires on ONE edge only.



Passing Events (1-bit)

Edge Detector types

- Four useful variants: rising/falling x active-high/active-low.



Passing Events (1-bit)

What got Solved

- Using synchronizers:
  - **Data corruption is partially solved** — only for one-bit signals
  - **Data incoherence is solved** — one synchronizer propagates to all the 2nd domain blocks with the same settled value
  - **Data loss is solved** — using pulse width $\geq 1.5 * \mathrm{Tclk}_2$
  - **Data duplication is solved** — using pulse and edge Detectors
  - **Chip burning is solved** — We limited the metastability propagation between the synchronizers and reduced its occurrence frequency

Multi-bit: Gray Coding

Multi-bit: Gray Coding

Gray Code

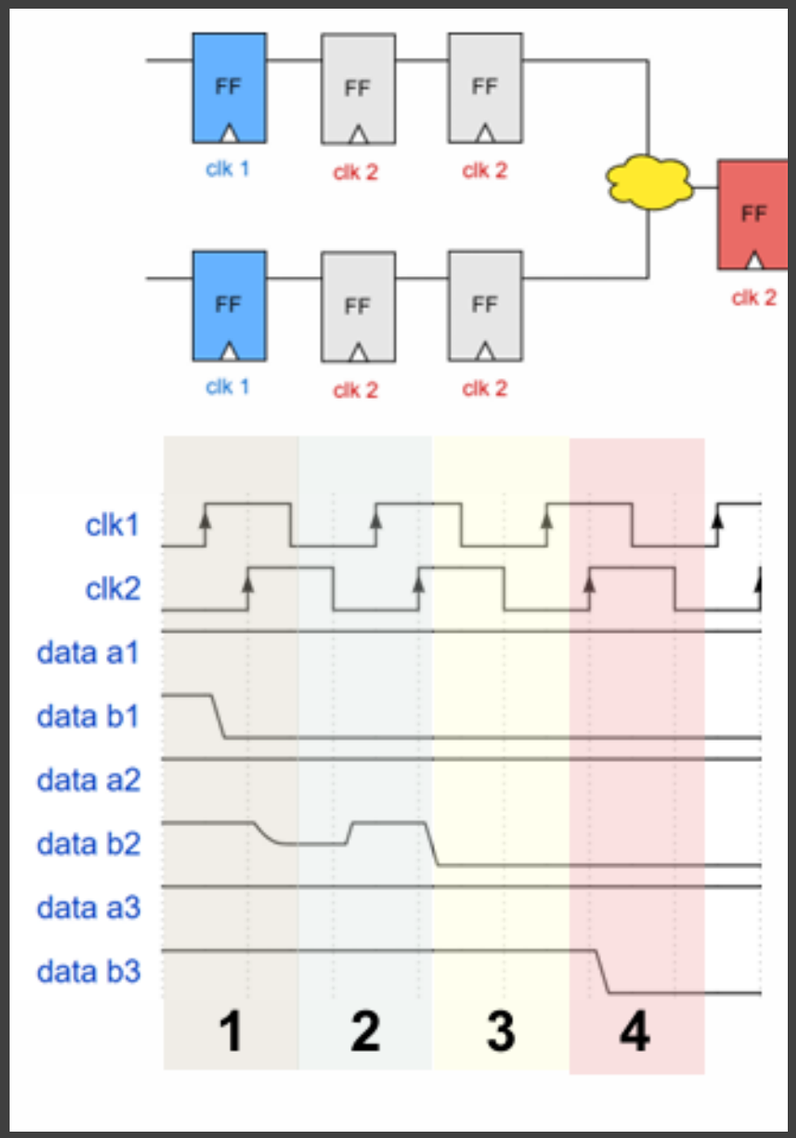
- Gray code: adjacent values differ in EXACTLY ONE bit.
- Gray coding is used when we send the data in sequence like counters or FSMs.
- Crossing a gray-encoded value can then only yield the OLD value or the NEW value as only one-bit changes with each transaction — never an illegal mixture.

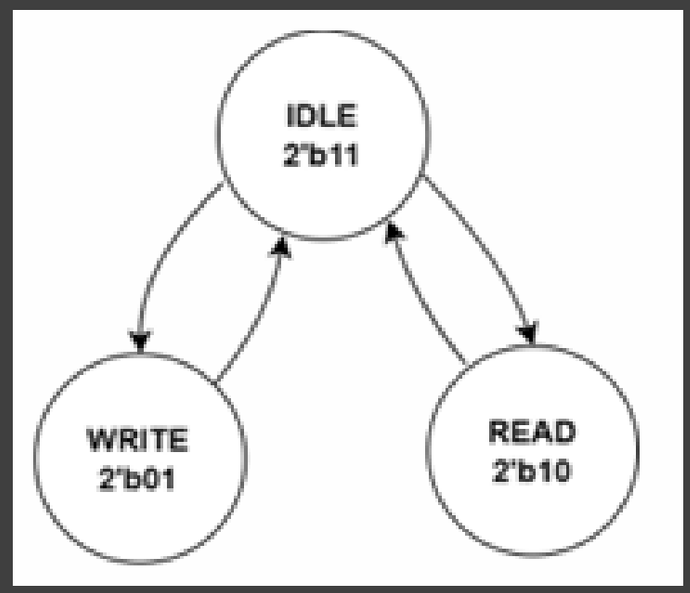
Decimal	0	1	2	3	4	5	6	7	8	9	10
Binary	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010
Gray	0000	0001	0011	0010	0110	0111	0101	0100	1100	1101	1111

Multi-bit: Gray Coding

Why Gray Works

- Same 2-bit FSM example, re-encoded so exactly one bit changes per transition (IDLE=11, WRITE=01, READ=00).
- One bit goes metastable; the other stays stable — the received value is the old or the new state, never a third state.





Multi-bit: Gray Coding

Binary <-> Gray Conversions

- **binary -> gray:** gray = (bin >> 1) ^ bin.
- **gray -> binary:** bin[i] = XOR-reduce(gray >> i) — done with a for-loop.
- Both are cheap, parameterized, and synthesize to XOR trees.

CDC - 05_binary_to_gray.v

```
1  // Binary -> Gray conversion.
2  // gray[i] = bin[i+1] ^ bin[i]; the MSB just passes through.
3  // One line, parameterized, synthesizes to an XOR tree.
4  module binary_to_gray #(parameter SIZE = 4) (
5      output logic [SIZE-1:0] gray,
6      input  logic [SIZE-1:0] bin
7  );
8      assign gray = (bin >> 1) ^ bin;
9  endmodule
10
```

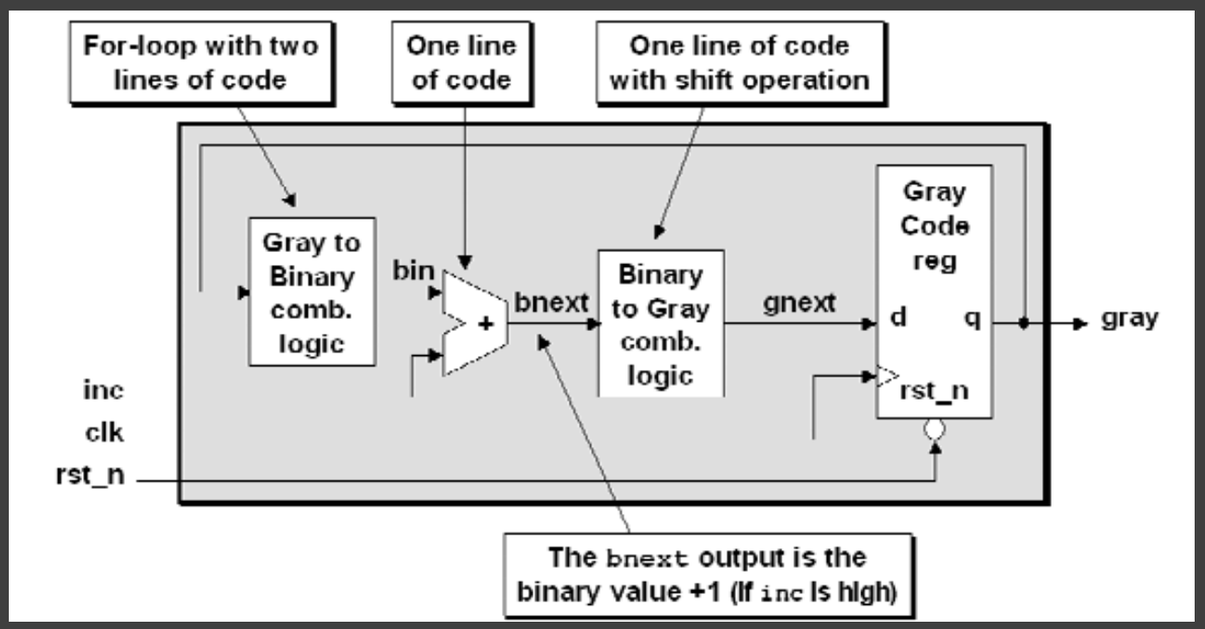
CDC - 06_gray_to_binary.v

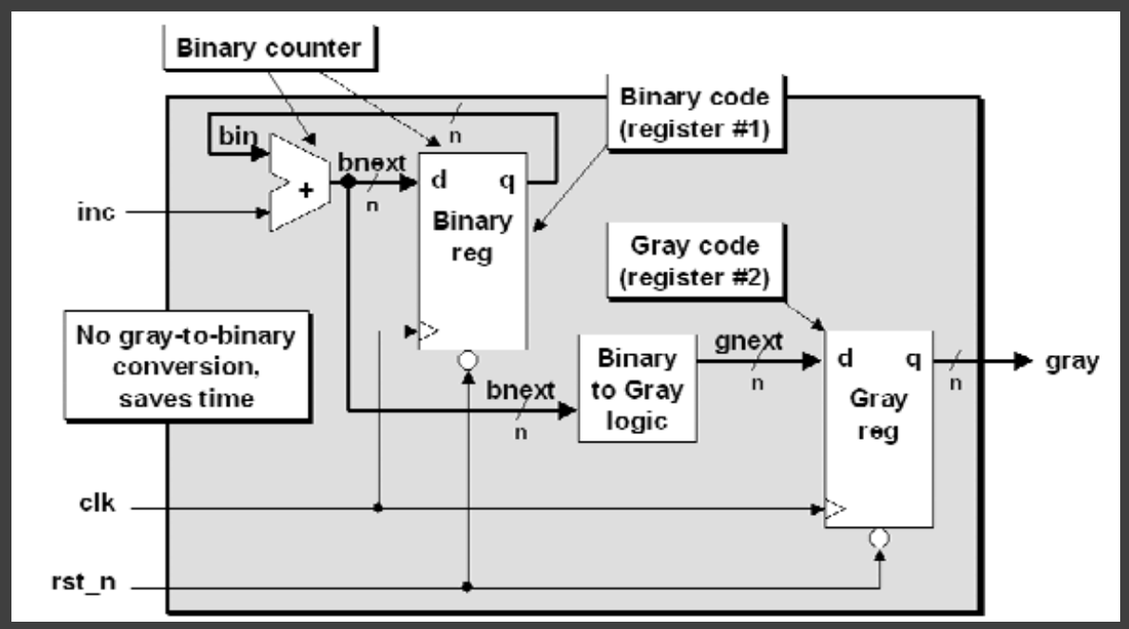
```
1  // Gray -> Binary conversion.
2  // bin[i] = XOR-reduce(gray >> i).
3  // The naive variable part-select (gray[SIZE-1:i]) is illegal Verilog -
4  // shifting the whole vector and reducing works and synthesizes well:
5  // the tool optimizes the constant-0 XORs away.
6  module gray_to_binary #(parameter SIZE = 4) (
7      output logic [SIZE-1:0] bin,
8      input  logic [SIZE-1:0] gray
9  );
10     always_comb
11         for (int i = 0; i < SIZE; i++)
12             bin[i] = ^(gray >> i);
13 endmodule
14
```

Multi-bit: Gray Coding

Gray Code Counters

- Counters are THE thing you gray-code across domains.
- **Style 1:** one gray register; convert gray->bin, increment, convert back.
- **Style 2:** binary register + gray register — twice the flops, shorter combinational path, faster.
- In both styles the computations happened to the binary values then it is converted to gray for crossing.
- The gray output must always be REGISTERED (no combinational settling on the crossing).





Multi-bit: Gray Coding

What Gray Can and Cannot Do

- **Works:** signals whose transitions are **SEQUENTIAL** and **predictable** — counters, FIFO pointers, some FSMs.
- **Does not work:** data values. You cannot know the next value, so you cannot guarantee 1-bit transitions (sending "562" then "177"?).

- **For arbitrary multi-bit DATA we need the MUX scheme, MCP formulation, handshake, or a FIFO — next sections.**

Multi-bit: Gray Coding

Example — Unsafe Multi-bit Crossing

- **A 2-bit FSM state crossing through two separate synchronizers — the classic WRITE/READ bug.**

- Gray coding and correct state sequencing will make the new value to be the old state or the new correct state and never go into a third or illegal state

CDC - 09_unsafe_multi_bit_crossing.v

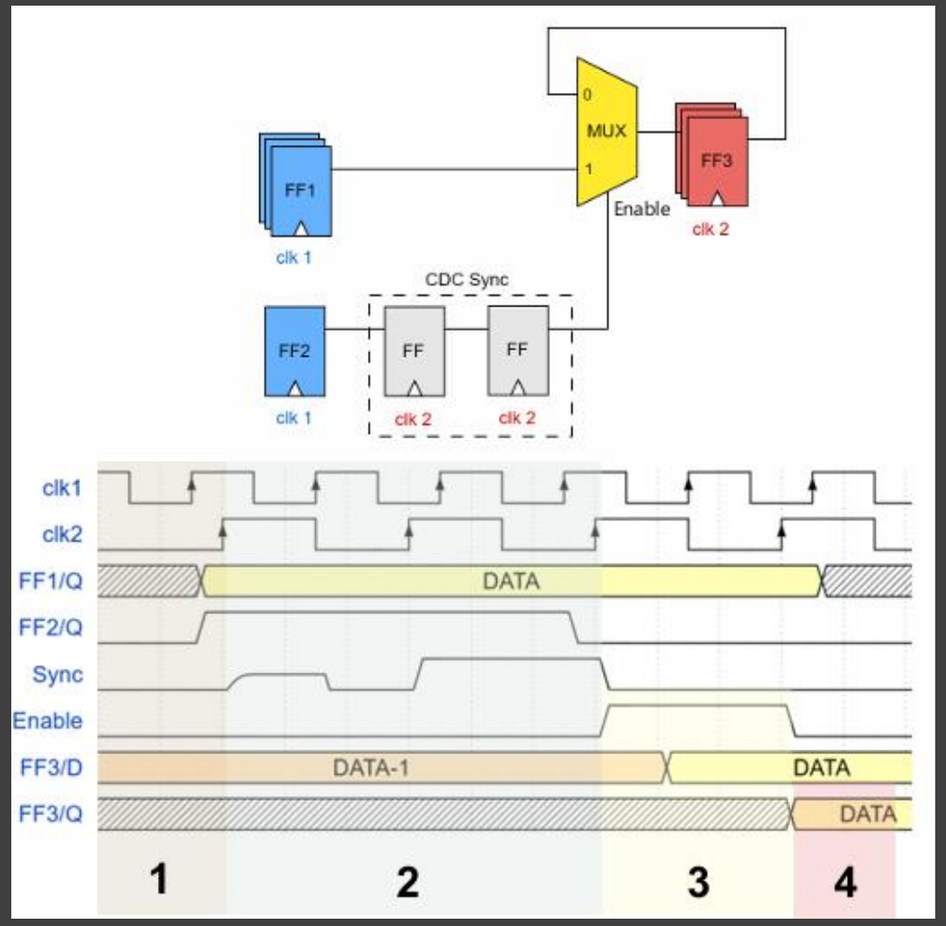
```
1  // UNSAFE: a 2-bit FSM state crossing through TWO independent synchronizers.
2  // The two bits can settle one cycle apart: state 2'b11 (WRITE) can be
3  // received as 2'b10 (READ) for a cycle - a legal but WRONG state.
4  // FIXES: converge to 1 bit in the tx domain, gray-code the value,
5  // or pass the bus as a group (MUX / MCP / FIFO).
6  module unsafe_multi_bit_crossing (
7      output logic [1:0] state_rx,
8      input  logic [1:0] state_tx,   // e.g. IDLE=2'b00, WRITE=2'b11, READ=2'b10
9      input  logic clk2, rst_n
10 );
11     logic sa1, sa2, sb1, sb2;      // two independent 2-FF synchronizers
12     always_ff @(posedge clk2 or negedge rst_n)
13         if (!rst_n) {sa2, sa1} <= '0;
14         else        {sa2, sa1} <= {sa1, state_tx[1]};   // bit 1
15     always_ff @(posedge clk2 or negedge rst_n)
16         if (!rst_n) {sb2, sb1} <= '0;
17         else        {sb2, sb1} <= {sb1, state_tx[0]};   // bit 0
18     assign state_rx = {sa2, sb2};  // can be a MIXTURE of old and new!
19 endmodule
20
```

Multi-bit: MUX & MCP

Multi-bit: MUX & MCP

MUX Recirculation Scheme

- Multi-bit DATA goes DIRECTLY to a MUX in the rx domain — unsynchronized.
- A 1-bit enable goes through the 2-FF synchronizer and opens the MUX (FF3 select) when the data is ready.
- While the enable is 0, FF3 recirculates its own output — nothing async reaches its D pin.
- The enable signal is sync with clk2 so no metastability happens for the data signals



CDC - 10_mux_synchronizer.v

```v
// MUX recirculation synchronizer.
// * DATA crosses UNSYNCHRONIZED into the rx-domain mux - by design
// * a synchronized 1-bit enable opens the mux only when data is stable
// * while en=0 the capture flop recirculates its own output, so no async
//   edge ever reaches its D pin (and STA times the enable path clk2->clk2)
module mux_synchronizer #(parameter WIDTH = 8) (
    output logic [WIDTH-1:0] data_rx,
    input  logic [WIDTH-1:0] data_tx,   // MUST stay stable while en crosses
    input  logic             en_tx,     // tx-domain enable (level)
    input  logic             clk2,
    input  logic             rst_n
);
    // 2-FF synchronizer on the enable
    logic q1, en_sync;
    always_ff @(posedge clk2 or negedge rst_n)
        if (!rst_n) {en_sync, q1} <= '0;
        else        {en_sync, q1} <= {q1, en_tx};

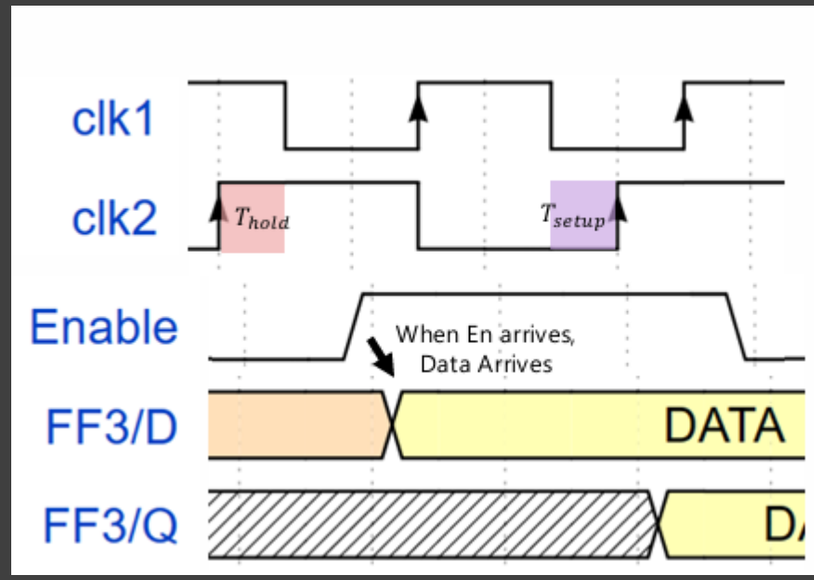
    // gated capture: recirculates while en_sync is low
    always_ff @(posedge clk2 or negedge rst_n)
        if (!rst_n)        data_rx <= '0;
        else if (en_sync)  data_rx <= data_tx;
        // implicit else: hold (recirculate)
endmodule

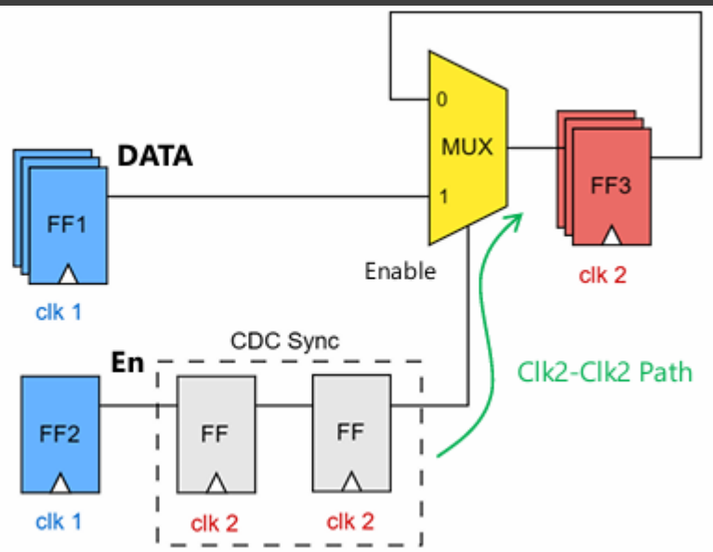
```

Multi-bit: MUX & MCP

Why It Works

- The data only moves when the **synchronized** enable opens the MUX.
- That enable-to-FF3 path is a clk2 -> clk2 path — fully analyzable by STA.
- **Assumption:** the data must already be STABLE in front of the MUX before the enable arrives.
- Enforce it with a ***max_delay*** constraint on the data path (Part: Timing Constraints).

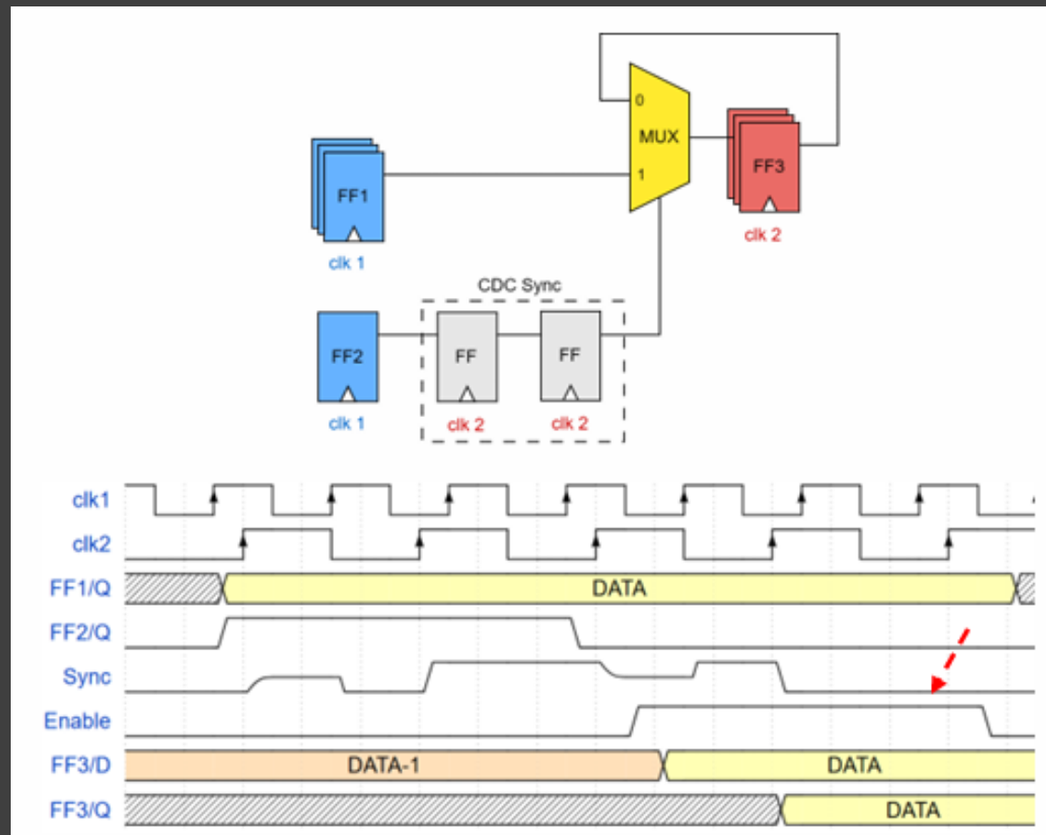


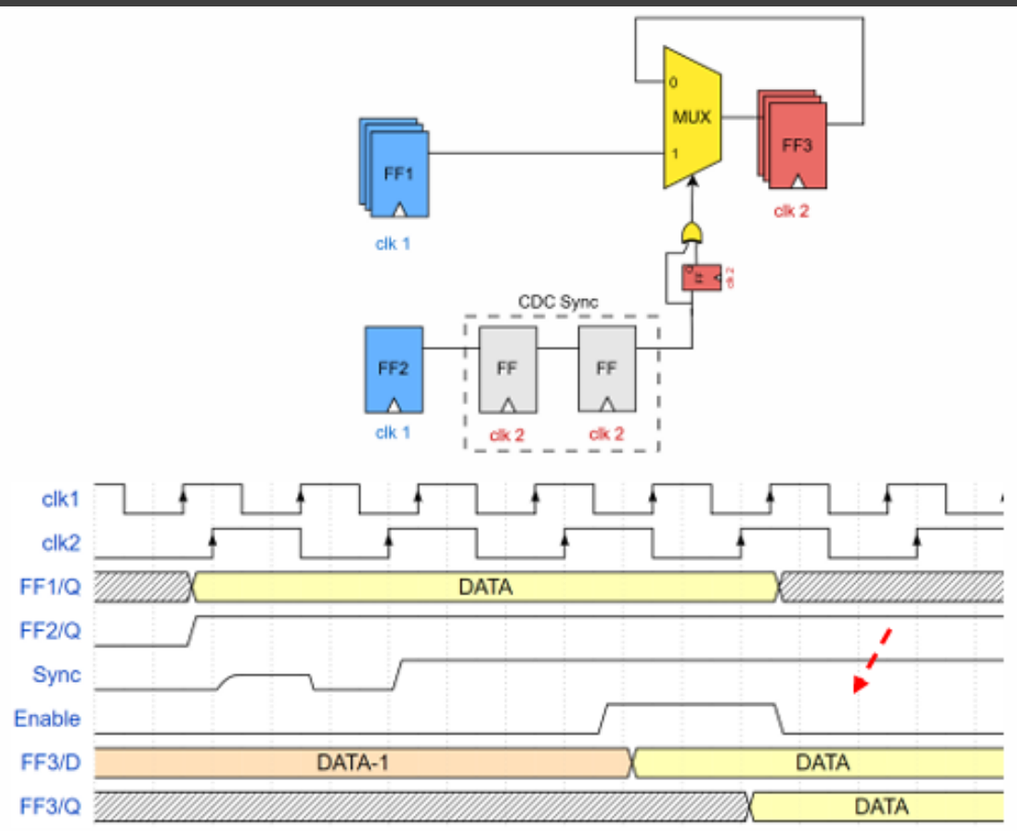


Multi-bit: MUX & MCP

How Long Must the Data Be Held?

- Worst case: syncs settle wrong, then right — the enable can arrive late and stay high longer.
- Data must not change while the enable is high, or the MUX passes **async** data again — CDC problem can.
- Data must be ready before enable gets high.
- **Hold the data for N + 2.5 clk2 cycles** (N = number of sync stages).
- **With a pulse synchronizer:** N + 1.5 cycles is enough (enable high for only 1 cycle).

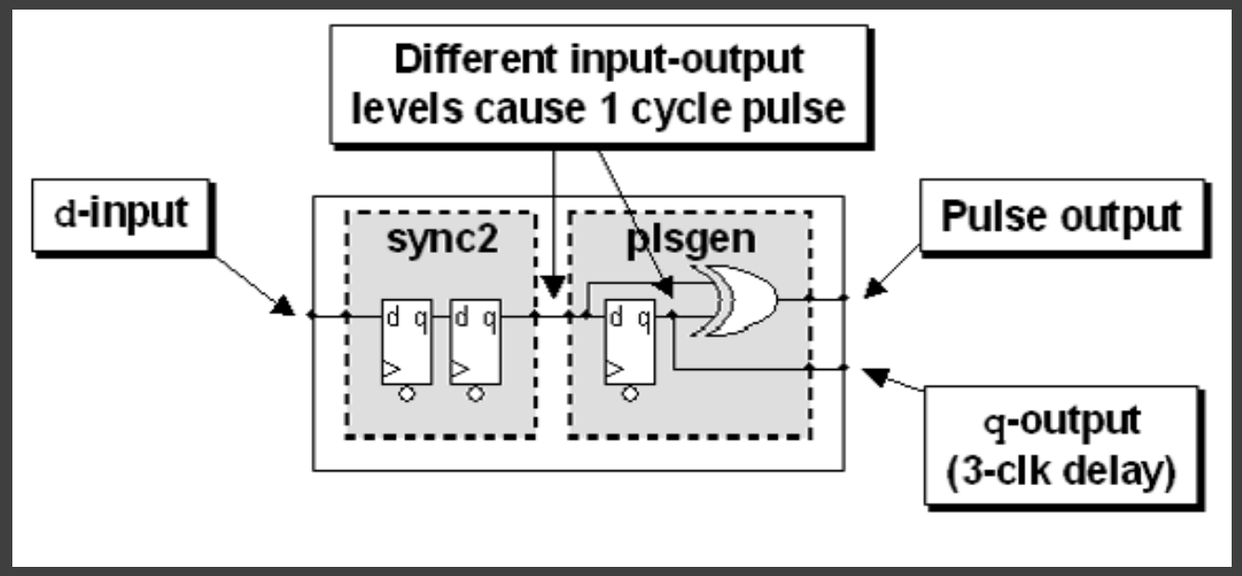


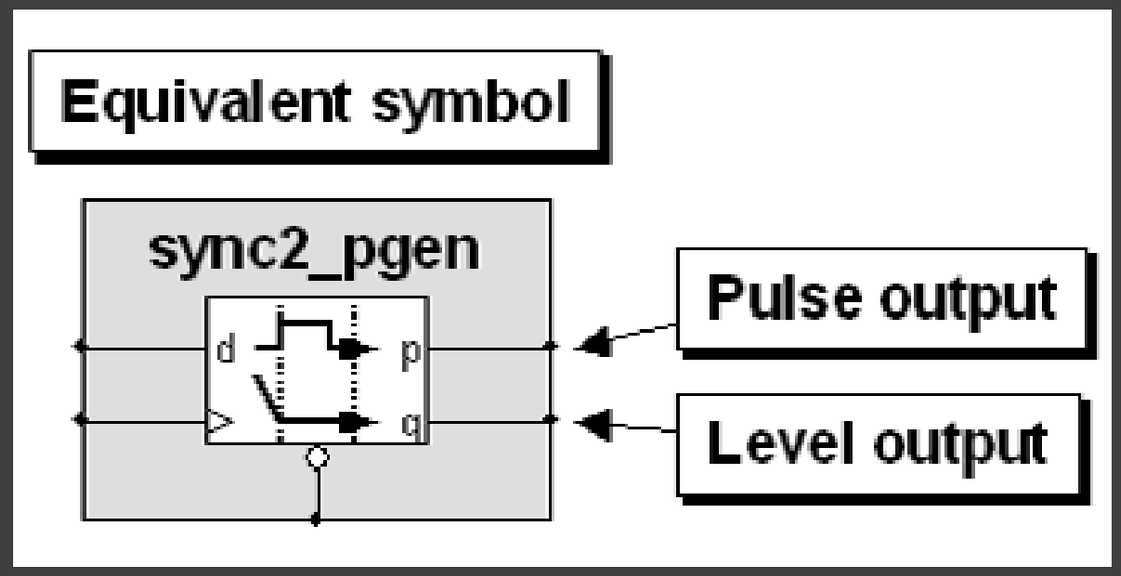


Multi-bit: MUX & MCP

MCP Formulation

- **Multi-Cycle Path formulation:** send unsynchronized data + a synchronized enable **TOGETHER**; hold the data for multiple rx cycles.
- The enable is a **TOGGLING** signal passed through a synchronized pulse generator — polarity does not matter.
- Same scheme as the MUX recirculation, seen from the timing-constraint side: the data path is a false-path/multi-cycle path by construction.

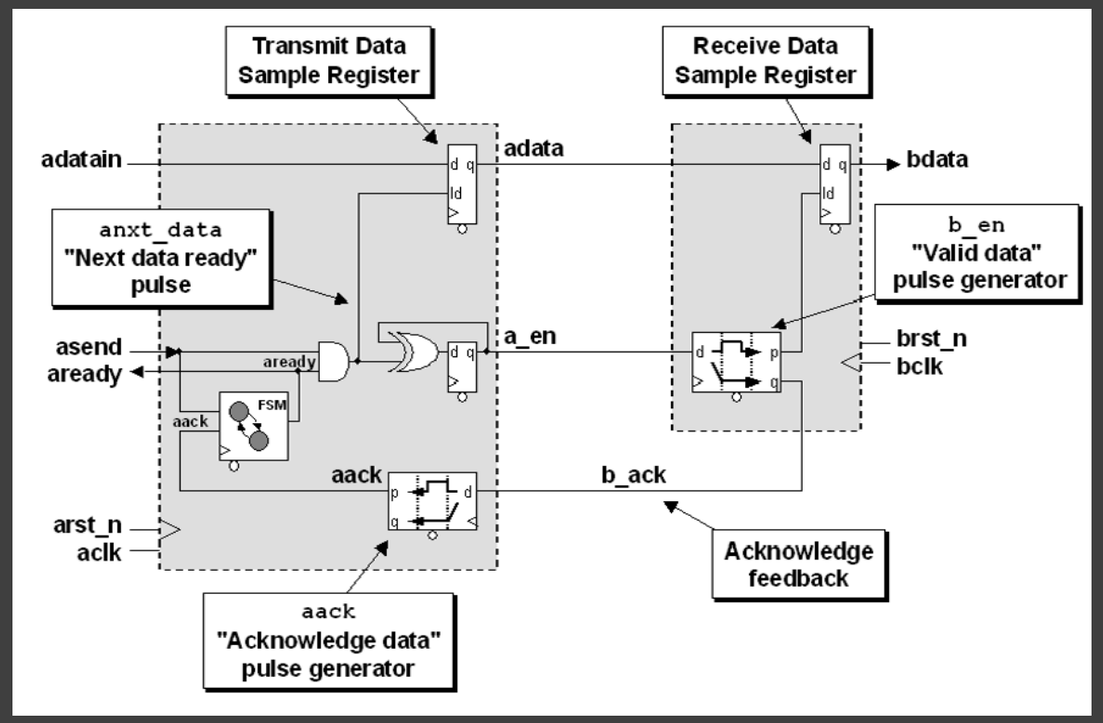




Multi-bit: MUX & MCP

Closed Loop — MCP with Acknowledge

1. When the TX is ready to send data it sample the ***aready*** signal then raises the ***asend*** signal and push the new data in the ***adatain*** signal.
2. Then the TX sends the data ***a_en*** flag becomes high after 1 clock cycle and a small READY-BUSY, 1-state FSM block becomes BUSY to make the ***aready*** signal low.
3. Then the MCP generate a pulse ***b_en*** signal to generate the ***bdata*** with the new value synchronized with the b_clock “after three clock cycles”; and then the b block generate ***b_ack*** to the a block to indicate the successful data sampling.
4. The last step is that the FSM becomes ready and makes the ***aready*** signal high again then the loop repeats



Multi-bit: MUX & MCP

The Bandwidth Problem

- MUX/handshake schemes hold the data stable and wait — the sending domain cannot push a new word every cycle.
- Fine for configuration and control. Useless for streaming data.

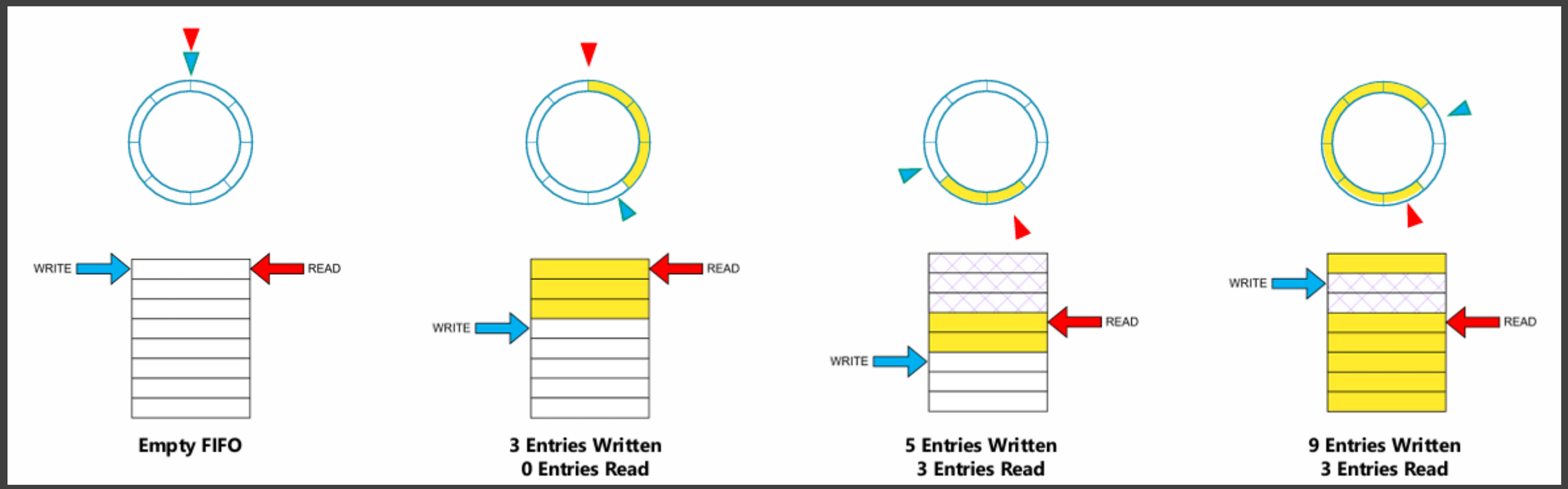
- **For high throughput we need a FIFO: write on clk1, read on clk2, buffers in between.**

Asynchronous FIFO

Asynchronous FIFO

Circular FIFO Structure

- A fixed-size buffer with a write pointer and a read pointer; both wrap around to 0 at the end.
- Write side pushes on wclk; read side pops on rclk — completely independent.
- **FIFO = the standard answer for multi-bit data CDC.**



Asynchronous FIFO

Empty and Full Conditions

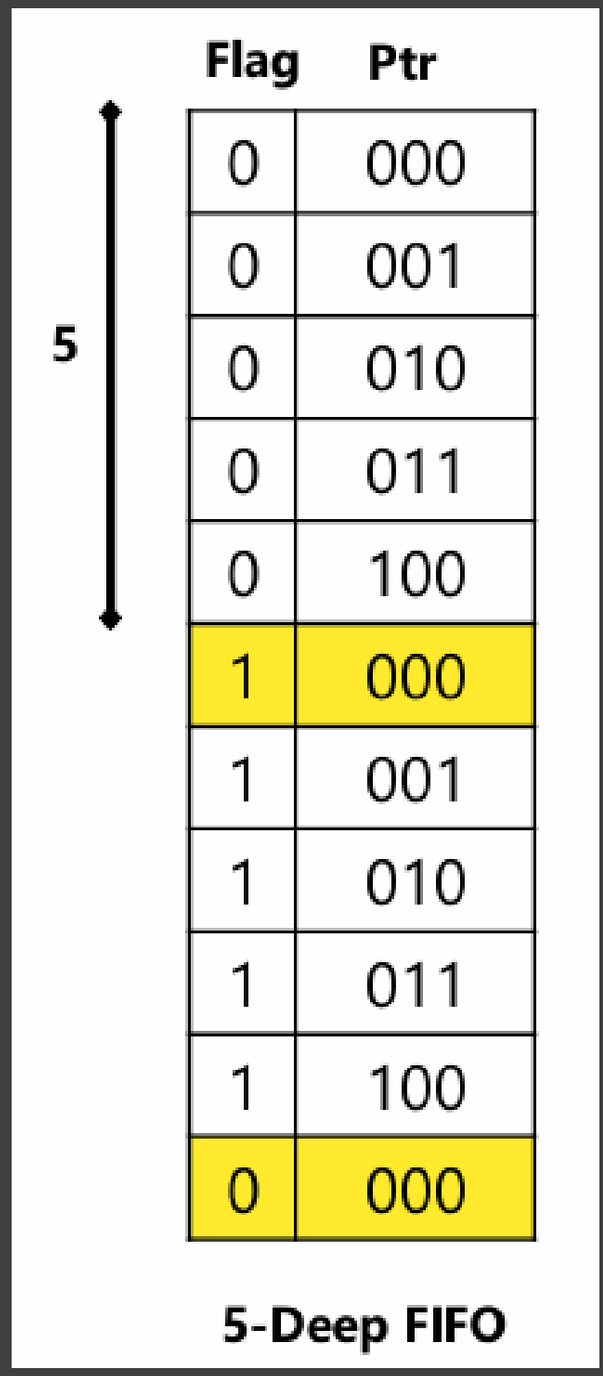
- Problem: equal pointers means empty OR full — indistinguishable.



Asynchronous FIFO

The Wrap Solution

- **Solution:** add a wrap flag (or use the pointer MSB as the flag).
- **Empty:** pointers equal AND wrap flags equal.
- **Full:** pointers equal AND wrap flags differ.
- Power-of-2 depth: the pointer naturally overflows to 0 and toggles its MSB — free wrap flag.

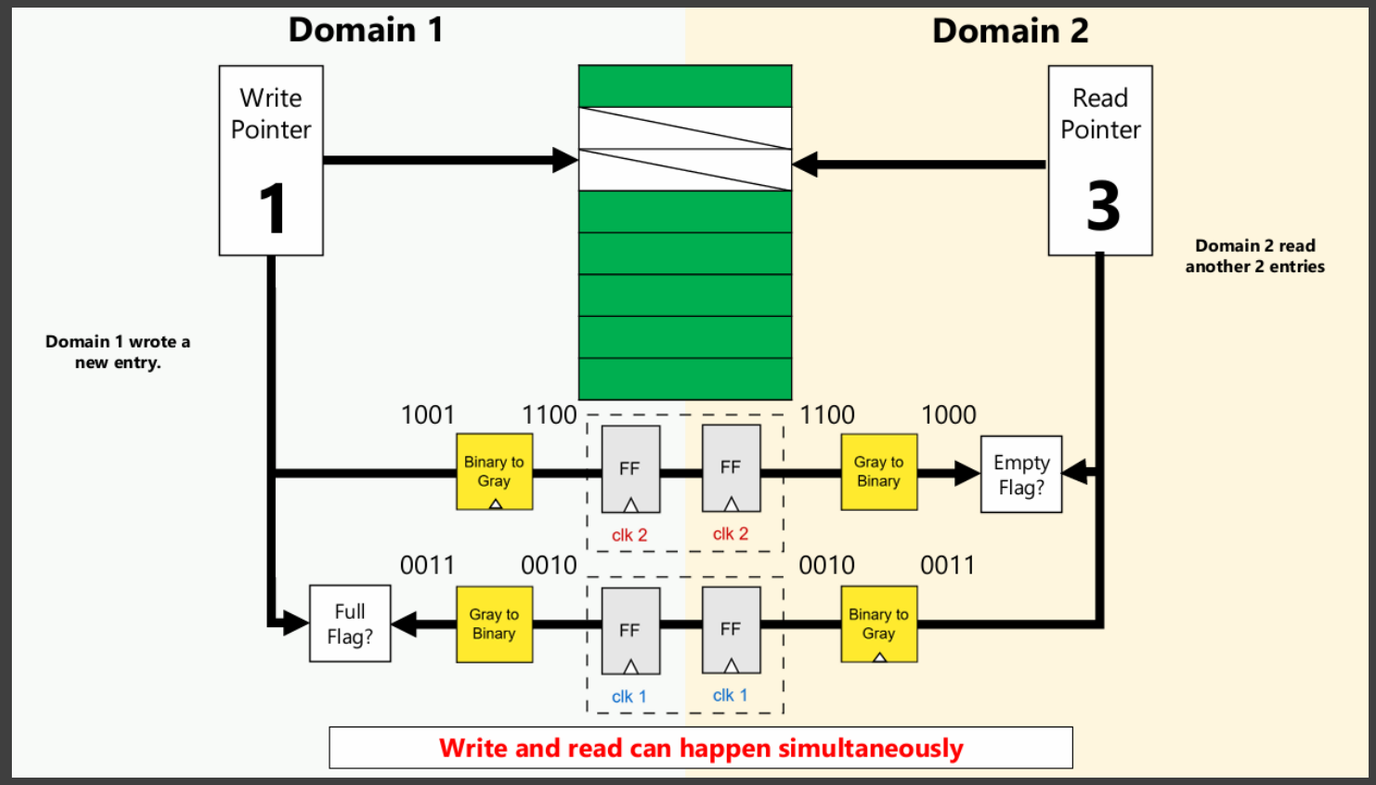


5-Deep FIFO

Asynchronous FIFO

CDC FIFO Architecture

- Pointers are gray-encoded, REGISTERED, then synchronized into the OTHER domain (2 FFs each way).
- Gray pointers guarantee the other side always sees a valid (old or new) count.
- FIFO flags are pessimistic (write side sees a delayed read pointer and read side sees a delayed write pointer) — safe by design.
- Reads and writes happen simultaneously in their own domains.



Asynchronous FIFO

FIFO Depth Calculation

- $\boldsymbol{Depth = Burst * (1 - \frac{f_{read}}{f_{write}})}$ **for the simple no-idle case.**
- Idle cycles between writes slow the writer down — effectively lowering fwrite, shrinking the needed depth.
- **Example 1:** fwrite=400MHz, fread=150MHz, burst=120 -> depth 75 -> round up to 128 (power of 2).
- **Example 2:** same, but 1 idle cycle per write (fwrite/2 effective) -> depth 30 -> 32.
- Round to a power of 2 so the MSB-wrap trick works.

Timing Constraints

Timing Constraints

CDC Paths and STA

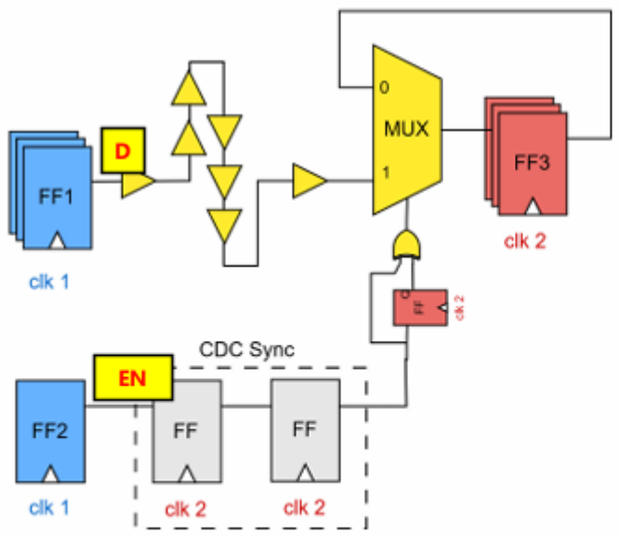
- By construction, CDC data paths are never timed as normal paths (async clocks, unknown phase).
- But the tools still need to be **TOLD**, and some paths (MUX enable, gray buses) must still be constrained.
- The next slides: what happens with the wrong constraint — and the right one.

Timing Constraints

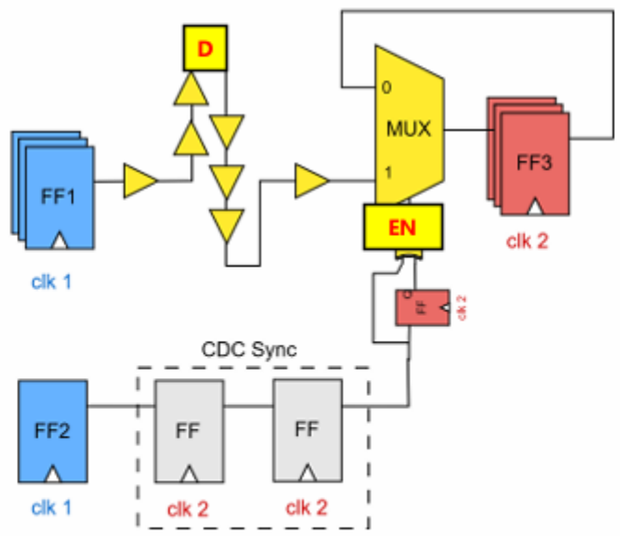
Wrong #1 — Blanket False Path

- set_false_path tells the tool the path does not matter — it then relaxes it: long routes, slow cells, big buffers.
- **MUX scheme broken:** data takes a huge detour and arrives AFTER the enable — async data passes the open MUX. Metastability.

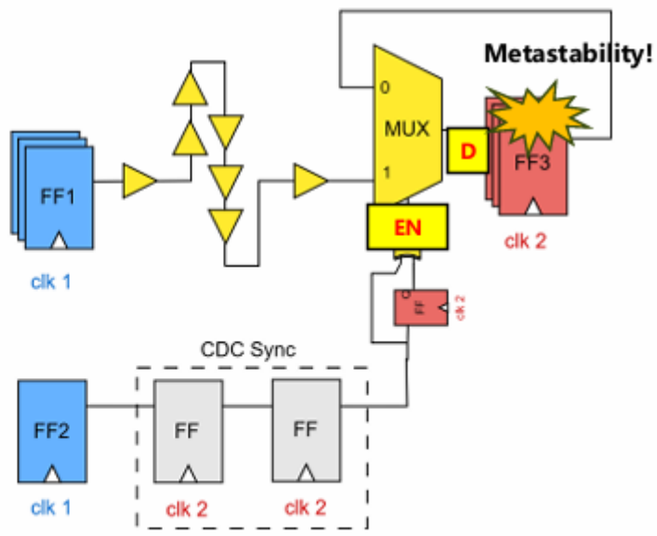
1 **Now let's consider the faulty behavior with a false path constraint applied[1]**
- The data goes to the Rx domain MUX
- The enable goes to the Rx domain through FF synchronizers



2
- The data takes a long time to reach the MUX due to the logic delay.
- The enable reaches the MUX and opens the gate



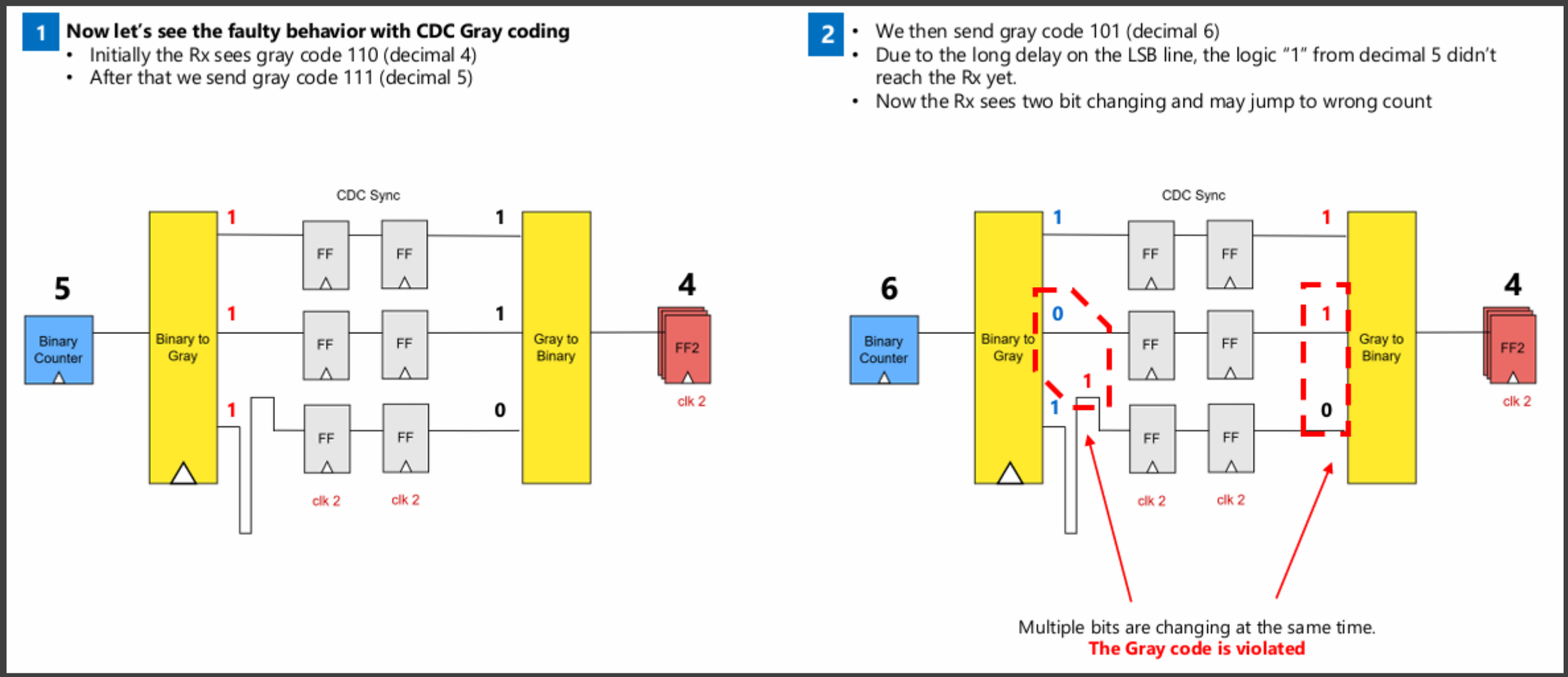
3
- After some time, the data arrives at the MUX after the enable.
- The data is a domain 1 signal arriving at domain 2 FF. Metastability happens



Timing Constraints

Wrong #1 — Blanket False Path

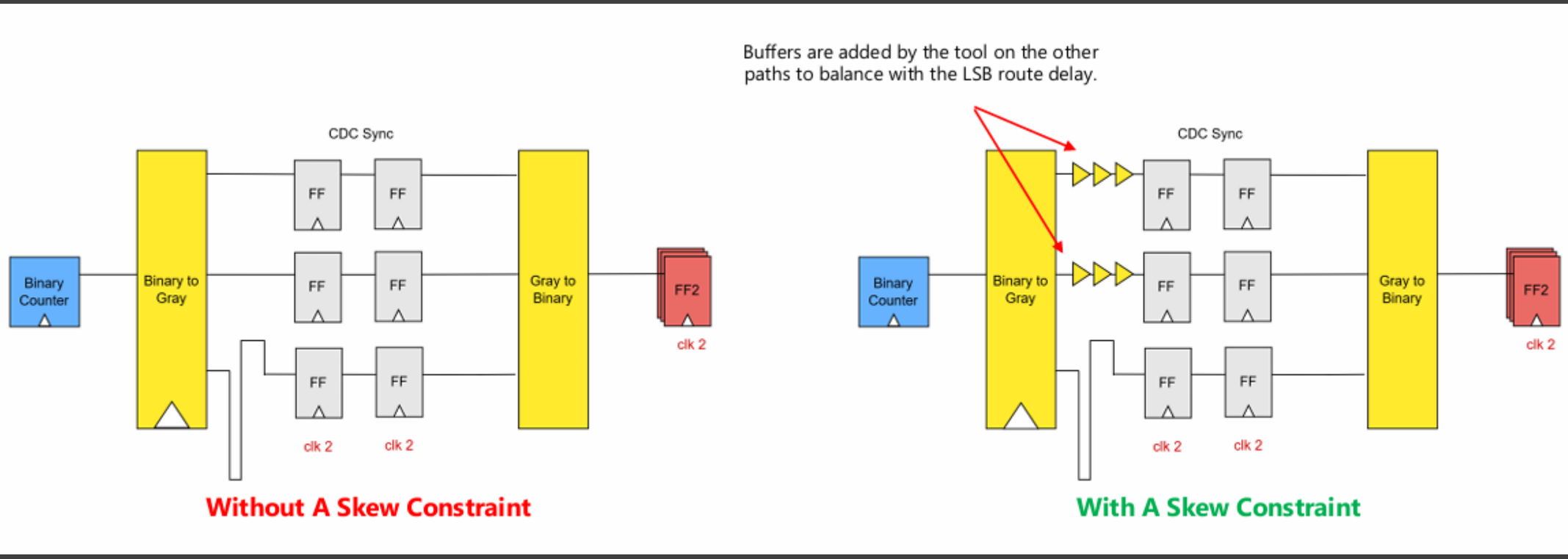
- **Gray scheme broken:** one bit is delayed so much that two bits change simultaneously at the receiver — the gray guarantee is gone.



Timing Constraints

Skew Constraints

- Constrain the arrival DIFFERENCE between bits of a group (e.g. <= 3ns across a 4-bit gray bus).
- The tool balances the bus by buffering the fast bits up to the slowest one.
- **Downside:** manual work — every CDC group needs its own skew constraint. Fine for a few groups, painful for hundreds.



Timing Constraints

Best — Max Delay Constraint

- **set_max_delay <small> -from CLK1 -to CLK2 — one line covers ALL crossings between the two domains.**
- Use the worst-case (smallest) relevant period as the value; tighten so CDC structures cannot break.
- Depending on the tool, first false-path the setup check, then apply max_delay (max_delay does not always override setup).
- Groups that are still too tight? Fall back to individual skew constraints.

CDC - 14_cdc_constraints.tcl

```tcl
################################################################################
## CDC timing constraints (PrimeTime style) - see "Timing Constraints".
################################################################################

## 1) NEVER blanket false-path the CDC paths - the tool relaxes them and
##    the MUX / gray structures break (enable overtakes data, bits drift):
# set_false_path -from [get_clocks clk1] -to [get_clocks clk2]   ;# WRONG

## 2) Best practice: max_delay on ALL paths between the two domains.
##    Use the smaller (worst-case) of the two periods as the value:
set_max_delay 4.0 -from [get_clocks clk1] -to [get_clocks clk2]
set_max_delay 4.0 -from [get_clocks clk2] -to [get_clocks clk1]
## Tool note: some tools need the setup check false-pathed FIRST so that
## max_delay governs - check your user guide.

## 3) MUX-synchronizer data path: data must be stable BEFORE the enable
##    arrives at the mux. A max_delay small enough enforces exactly that.

## 4) Gray-coded pointer buses: keep the bits arriving together.
##    Modern alternative to per-group skew constraints:
set_max_skew -exclude_pin [get_ports clk] 0.500 [get_nets {gray_ptr[*]}]
## (or hand-written per-group skew constraints - painful for many groups)

## 5) Declare the clocks asynchronous so normal setup/hold is skipped:
set_clock_groups -asynchronous -group {clk1} -group {clk2}
## NOTE: this already cuts CDC timing in most tools. It works together
## with (2) because explicit max_delay constraints override the cut in
## PrimeTime - VERIFY this in your own tool before trusting it.

```

CDC Verification

CDC Verification

Static CDC Tools

- Structural analysis: find every crossing, check it against the CDC rules, classify severity (warning / critical / error).
- They catch: unsynchronized signals, combinational logic in front of syncs, divergence, reconvergence, missing constraints.
- Run them early and often — a CDC report is a to-do list, not a formality.

CDC Verification

Simulation and Formal

- RTL simulation rarely hits the metastability window — not a CDC sign-off method.
- **Gate-level sim:** synchronizer flops go X; strategies: synchronizer cells with X-clean-up, force notifiers, random delay injection.
- **Formal/meta-stability injection:** the tool asserts the first sync output arbitrarily and proves the design still works (also covers the +/-1 cycle uncertainty).
- Combine: static structural checks + formal CDC + targeted simulation of the crossing protocols.

Summary

Summary

Which Technique When?

What you are crossing	Technique
Level signal (quasi-static, config)	2-FF / 3-FF synchronizer
Single event (pulse)	Toggle in tx + pulse generator in rx
Multi-bit SEQUENTIAL value (counter, pointer)	Gray coding
Multi-bit data, low rate	MUX recirculation / MCP + handshake
Multi-bit data stream, high rate	Asynchronous FIFO (gray pointers)
Any CDC scheme	max_delay constraints, never blanket false paths

Summary

Concerns Tracker — Final

Concern	Solved by
Data corruption	Fault-tolerant protocols (valid/ready), gray coding
Data incoherence	Synchronizers + single-sync rule (no divergence)
Data loss	Pulse width rule / toggle sync / FIFO
Data duplication	Pulse generator / toggle sync
Chip burning	Synchronizer isolation + high MTBF

References

References

Papers and Articles

[1] Amr Adel Mohammady, "Clock Domain Crossing" series

[2] C. Cummings, "Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog", SNUG Boston 2008.

[3] ChipVerify, "Clock Domain Crossing" article — chipverify.com/rtl-synthesis/clock-domain-crossing.

FRIENDLY REMINDER

Israel has killed more than 13,000 children in Gaza since October 7 while others are suffering from severe malnutrition and do not “even have the energy to cry”,
says the United Nations Children’s Fund (UNICEF).

“Thousands more have been injured or we can’t even determine where they are. They may be stuck under rubble … We haven’t seen that rate of death among children in almost any other conflict in the world”
-UNICEF Executive Director

Save The Palestinian Children

Thanks